Journal of Cloud Computing

REVIEW Open Access

Matrix protocol: a comprehensive systematic mapping study

José A. P. Martins[1], Paulo A. L. Rego[1], José A. F. de Macêdo[1], Francisco Airton Silva[2*] and Vinícius Lagrota[3]

Abstract

The Matrix protocol is gaining significant traction in the domain of decentralized communication, offering robust features like end-to-end encryption, federated server architecture, and seamless interoperability. As more applications and platforms adopt this protocol, understanding its performance and security implications becomes increasingly crucial. Evaluating the Matrix protocol's adoption and its impact on communication systems is challenging due to its decentralized nature, diverse implementation scenarios, and varying security requirements. Existing studies provide fragmented insights, lacking a comprehensive overview of how the Matrix protocol performs in real-world settings, especially concerning security, scalability, and interoperability. This paper presents a systematic mapping study, which rigorously examines 156 papers, narrowing them down to 21 primary studies that directly engage with the Matrix protocol. The study categorizes the research based on the relationship with the Matrix protocol, the main area of focus, the contributions made, and the specific features of the Matrix protocol explored. By employing visualizations and detailed cross-analysis, the study reveals key trends, research gaps, and provides a foundational understanding of the protocol's strengths and limitations in decentralized communication. This study aims to guide future research efforts, advocating for more structured evaluation frameworks to enhance the understanding and deployment of the Matrix protocol in diverse communication environments.

Keywords Systematic mapping study, Matrix protocol, Matrix basic concepts, Area of study, Research contributions

Introduction

Context and motivation

According to the Matrix.org Foundation, as a concept, *"the Matrix is an open protocol for decentralized, secure communications, developed to enable interoperability between different communication systems"* [1]. As digital communication continues to evolve, the importance of secure and private interactions has never been more significant [2]. Matrix offers a robust framework that prioritizes user control and data sovereignty, allowing individuals and organizations to maintain ownership of their communications. This is especially important in a world where centralized platforms often pose risks related to data breaches and unauthorized surveillance. The protocol's unique ability to provide secure, real-time communication across a federated network of independent servers makes it highly adaptable to the needs of modern distributed systems. This adaptability is further enhanced by Matrix's support for a wide range of communication types, including instant messaging, voice-over-IP (VoIP), and even video conferencing. Unlike traditional centralized messaging platforms, Matrix enables interoperability between different services, allowing users to communicate seamlessly across various networks [3]. This interoperability is a key differentiator

*Correspondence:
Francisco Airton Silva
faps@ufpi.edu.br
[1]Federal University of Ceará (UFC), Fortaleza, Ceará, Brazil
[2]Federal University of Piauí (UFPI), Picos, Piauí, Brazil
[3]Research and Development Center for Communication Security (CEPESC), Brasília, Distrito Federal, Brazil

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

that positions Matrix as a forward-thinking solution in the landscape of digital communication. By breaking down the silos typically created by proprietary messaging systems, Matrix fosters a more connected and inclusive communication environment, where users are not restricted by the boundaries of specific platforms.

As a result of these strengths, the popularity of the Matrix protocol is rapidly increasing, with its adoption spreading across a variety of applications, from secure messaging and VoIP services to decentralized collaboration tools [4]. This widespread adoption is not only a testament to the protocol's technical capabilities but also reflects a growing awareness of the importance of privacy and security in digital communication. More organizations and communities are recognizing the need to replace from centralized systems—that can be vulnerable to breaches and data leaks—by the Matrix due to its ability to offer more security and resiliency [5]. The versatility of the Matrix protocol is another key factor driving its popularity. It is not limited to just one type of application or use case but can be adapted to meet a wide range of communication needs. The Matrix protocol has been widely adopted by governments, companies, and organizations due to its ability to provide decentralized, secure, and interoperable communication. A notable example is the French government, which developed Tchap, a platform based on Matrix, to ensure digital sovereignty and security in communications between public officials [6]. Another case is the NHS in the United Kingdom, which uses Matrix to establish a secure communication network compliant with privacy regulations such as the GDPR in its medical operations [7]. Cities like Berlin have also adopted the protocol to improve the efficiency and resilience of governmental communications [8].

Research problem

A federated architecture, like Matrix, enables decentralized and interconnected server operations. In this model, each server maintains autonomy while exchanging information with other network servers, creating a distributed system without a central control point. Combined with the necessity for interoperability across various platforms, makes it difficult to perform comprehensive evaluations that can guide the development and optimization of Matrix-based systems [9]. Existing research often provides fragmented insights, lacking a unified framework for assessing the protocol's performance under different conditions and usage scenarios. As organizations increasingly adopt Matrix for critical communication needs, the absence of a structured approach to evaluate its effectiveness and security hinders the ability to fully capitalize on its potential. The lack of standardized metrics and methodologies for assessing Matrix's performance in real-world deployments further complicates efforts to optimize its usage. This problem underscores the urgent need for a systematic mapping study that consolidates existing knowledge, identifies the critical aspects of Matrix evaluation, and provides a clear direction for future research. By addressing these challenges, this study aims to establish a foundation that can support more effective and informed use of the Matrix protocol in a wide range of applications.

Contribution

To address these challenges, this paper presents a comprehensive and focused systematic mapping study dedicated exclusively to the Matrix protocol—an area that has not yet been extensively explored in the literature. Rather than merely listing existing works, our study provides a structured classification and critical analysis of how the protocol has been adopted, evaluated, and discussed in academic contexts. The main contributions of this study include:

- A thorough identification and categorization of 156 papers, leading to the selection of 21 primary studies that directly engage with the Matrix protocol;
- A novel classification framework that maps each study in terms of its relationship with the protocol, research area, key contributions, and explored features;
- A detailed cross-analysis that reveals prevailing research trends, underexplored areas, and critical challenges that hinder the broader adoption and evaluation of Matrix;
- The provision of visual and tabular syntheses to support the understanding of the current research landscape and facilitate future comparative studies;
- An evidence-based discussion that informs and guides researchers and practitioners about directions for future work on decentralized, secure communication systems.

Outline

The remainder of this paper is organized as follows: Sect. 2 presents the Matrix protocol in details. Section 3 reviews related work, providing context and highlighting key contributions in the field. Section 4 details the methodology used in our systematic mapping study, outlining the steps taken to identify and analyze relevant literature. Section 5 presents the findings based on individual classifications, while Sect. 6 identifies the challenges encountered during the study and suggests future research directions. Finally, Sect. 7 concludes the paper with recommendations for further investigations in this area.

Overview of the Matrix protocol

Matrix protocol, an open standard for secure and decentralized communication, is gaining significant attention from academia, industry, and governments. Matrix builds upon existing components to establish an open protocol that empowers users to have full control over their data and communication. It is important to explore the historical context that influenced its development to understand its impact and usefulness. Additionally, we have examined the architecture that supports its distributed nature and the various use cases and applications that have emerged across different sectors. This section covers the evolution of the Matrix protocol and examines its technical framework. It also highlights the wide range of implementations demonstrating flexibility and adaptability in today's communication landscape.

Historical context

Matthew Hodgson and Amandine Le Pape are the cofounders of Matrix. Both initially worked at Amdocs, where they conceived Matrix to provide a more open and secure framework for real-time communication. The central idea was to create a decentralized communication infrastructure enabling interoperability between different messaging services. This approach aims to overcome the limitations of centralized protocols that create "communication silos" [1].

Table 1 summarizes the major changes in the Matrix protocol from its inception to the present year. It highlights significant milestones in the protocol's evolution, including version releases, security enhancements, adoption by governmental institutions, and the development of new features.

In May 2014, the team launched the first version of the Matrix protocol and the Riot client (later renamed Element), marking a significant step forward in decentralized communication. This initial release focused on interoperability and creating an open and decentralized communication network [6]. In July 2017, the Matrix.org Foundation was established as a non-profit organization to maintain and promote the development of the protocol as an open and free standard [1]. Later that year, E2EE was introduced into the protocol in October, significantly enhancing communications security on the Matrix network [1]. The year 2018 brought further innovations with the announcement of the beta version of the RiotX client in November. This was a complete rewrite of the Riot client, to provide a more modern and efficient user experience [6]. In June 2019, the Matrix protocol reached a significant milestone with the release of version 1.0 of its specification. This release solidified Matrix as a stable, production-ready solution, introducing security improvements, including detailed specifications for end-to-end encryption. Additionally, it formalized the protocol's APIs, such as the Client-Server API, Server-Server API, and Application Service API, establishing a clear standard for developers and integrators [1]. In December 2019, the RiotX client officially replaced the original Riot client, offering a faster and more secure experience [6].

In 2020, the protocol became more popular and adopted by various governmental institutions. This was especially notable in France, where Matrix was used to create "Tchap". Tchap is a secure communication system specifically designed for public officials [10]. In July of the same year, the Riot client was renamed to Element, consolidating its brand and market presence as the official client for the Matrix protocol [6]. Development continued in 2021 with the release of version 1.1 of the Matrix specification in February. This version introduced improvements in identity management and enhanced the robustness of server federation [1]. In October, the Spaces feature was introduced, allowing rooms to be organized into hierarchical groups, facilitating community management within Matrix [1]. In 2022, new versions of the protocol were released: version 1.2 in March, which brought optimizations in event synchronization and encryption enhancements [1], while version 1.3 in December focused on performance improvements and expanding federation capabilities [1]. The year 2023 saw significant updates. In May, version 1.4 was presented, focusing on greater compatibility with mobile devices and new security features [1]. In August, version 1.5 was released, bringing updates to the identity services API and support for new IoT (Internet of Things) use cases

Table 1 Chronological summary of changes in the Matrix protocol

Year	Historic Events
2014	Initial launch of the Matrix protocol and the first client, Riot (later renamed Element).
2017	Creation of the Matrix.org Foundation and introduction of end-to-end encryption (E2EE).
2018	Announcement of the beta version of RiotX, a major rewrite of the Riot client.
2019	Release of version 1.0 of the Matrix protocol specification and replacement of Riot with RiotX.
2020	Widespread adoption by governmental institutions, especially in France with the "Tchap" system. Riot renamed Element.
2021	Release of version 1.1 of the Matrix specification with improvements in identity management and server federation. Spaces feature introduced.
2022	Release of versions 1.2 and 1.3, focusing on event synchronization, encryption enhancements, and performance improvements.
2023	Releases 1.4–1.9, including compatibility improvements, moderation tools, and enhanced interoperability.
2024	Release of version 1.10–1.11 with security enhancements and refined user experience.

[1]. The protocol continued its evolution with the release of version 1.6 in June, which included enhanced support for AR and VR applications [1]. In August, version 1.7 improved interoperability with other decentralized communication networks, further strengthening Matrix's position as a versatile and secure solution for decentralized communication [1]. October 2024 saw the release of version 1.8, which focused on enhancing the scalability and performance of the Matrix network with optimizations for large-scale deployments and better database management. In December 2024, version 1.9 introduced advanced moderation tools to manage public and private rooms better and improve user identity management.

Finally, in February 2024, version 1.10 focused on security enhancements, including updates to encryption algorithms and additional protections against spam and abuse. In April 2024, version 1.11 was released, integrating new features to improve user experience, including refined user UI and more efficient communication protocols, further solidifying Matrix's position as a leading decentralized communication platform [1]. Having explored the historical development of Matrix, we now move on to examine the architecture and functionality that make it a robust decentralized platform.

Architecture and functioning of the matrix protocol

The Matrix protocol facilitates decentralized, secure, and interoperable communication across various devices and platforms. Its architecture relies on several core components and mechanisms that ensure efficient network data synchronization, user discovery, and message integrity.

General structure of the Matrix protocol

The general structure of the Matrix protocol is based on a federated architecture, where multiple independent servers, known as **homeservers**, communicate with each other to synchronize data and messages across the network. Each homeserver is an independent entity that can be run by anyone—from individuals to large organizations—allowing for a truly decentralized communication infrastructure. This architecture contrasts with traditional centralized models, where a single server or a cluster of servers controlled by a central authority handles all data and message routing.

In the Matrix ecosystem, each user is associated with a specific homeserver, their primary point of presence in the network. This homeserver stores the user's messages, profile data, contact lists, and other relevant information. When users register on the Matrix network, they choose a homeserver to register their accounts. All the user's data, including messages, media, and room memberships, are stored on this homeserver. Importantly, users on different homeservers can still communicate seamlessly because homeservers are designed to federate with one another, synchronizing data to ensure a consistent global state across the Matrix network [1]. Homeservers communicate using the **Server-Server API** (Federation API), which defines the protocols and methods for exchanging events, messages, and states between servers. This dual-API structure ensures a clear separation between client communication and server-to-server synchronization, contributing to the scalability and resilience of the network [1].

End-to-end message lifecycle and server synchronization

When a user sends a message in Matrix, the process begins on the client device. The message is first encrypted using the Olm protocol (for one-to-one communication) or Megolm (for group communication). In both cases, encryption occurs locally, and only the intended recipients hold the necessary keys to decrypt the message [1].

After encryption, the message is signed and sent to the sender's homeserver. The homeserver serializes it as a JSON event and assigns it a unique identifier. This event is stored and then propagated to other homeservers participating in the same room using the Server-Server API, which handles federation across the network [1].

Each homeserver that receives the event first verifies its digital signature, generated by the origin server using Ed25519 keys. If the signature is valid, the event is integrated into the server's local copy of the room's event history. Matrix structures this history as a Directed Acyclic Graph (DAG), where each event references one or more previous events. This model allows for asynchronous message propagation and enables each server to build a consistent causal structure of the room's evolution [3, 11, 12].

Conflicts may occur when events arrive out of order or in parallel, particularly under network delays or partitioning. In these cases, Matrix applies a deterministic state resolution algorithm. The algorithm uses attributes such as event depth, timestamps, and signature validity to determine the authoritative state of the room. This process allows all non-malicious homeservers to eventually converge on the same room state, without requiring centralized consensus [3, 11].

This mechanism ensures that Matrix supports decentralized communication with strong consistency properties. It guarantees that all participants, regardless of which server they connect to, will see a coherent and verifiable conversation history [13, 14].

Users and the client-server API

Figure 1 illustrates the basic configuration of a federated Matrix network, in which each client is connected to a dedicated server. This setup allows clients to communicate within their respective servers while also enabling cross-server interactions through federation. Each server

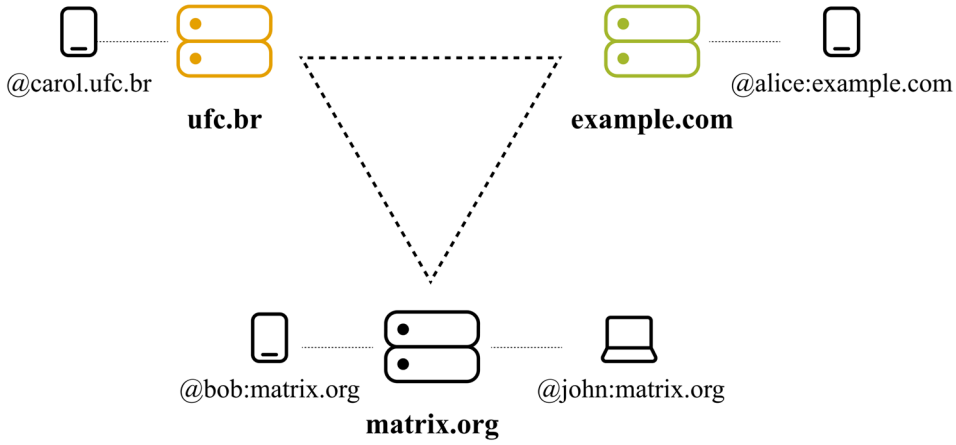


Fig. 1 Clients connected to homeservers. The servers are federated together

Table 2 Essential functions of the client-server API in the Matrix protocol

Function	Description
Continued on the next page	
User Authentication	Verifies user credentials to allow access to the network.
Message Sending and Receiving	Sends messages to and receives messages from the homeserver.
Room Management	Manages room creation, membership, and permissions.
Profile and Presence Management	Updates user profiles and displays user status.
E2EE	Encrypts messages for secure communication.
Push Notifications	Alerts users of new messages or events.
Media Uploading	Allows media file uploading and sharing.
Content Filtering	Controls incoming content visibility.

acts as an independent node, managing its own user data and handling message routing, yet remaining interconnected with other servers in the network to facilitate seamless communication across the federation.

The Matrix protocol places users at its center, interacting with the network primarily through clients—applications such as Element [6], FluffyChat [15], or Hydrogen [16]. These clients provide a user-friendly interface that allows users to send and receive messages, join rooms, manage their profiles, and communicate with other users on the Matrix network. The communication between users and the Matrix network is facilitated by the Client-Server API, which is a critical component that defines the methods and protocols used to connect client applications to homeservers.

The Client-Server API enables various essential functions that allow users to interact seamlessly with the Matrix network. Table 2 outlines the essential functions of the Client-Server API in the Matrix protocol. It highlights how client applications interact with homeservers to facilitate secure and efficient communication.

The Client-Server API manages the authentication process when a user logs in or registers on the Matrix network. It verifies the user's credentials against the homeserver database, ensuring proper authentication and authorization for accessing the network. Additionally, users can send messages to individuals or groups through their client applications. The Client-Server API transmits these messages to the user's homeserver, which then propagates them to other homeservers through federation. Incoming messages are similarly retrieved from the homeserver and displayed in real-time on the user's client. Users can manage their participation in rooms by creating, joining, or leaving rooms, with the API offering tools for managing room memberships, permissions, and administrative tasks such as inviting or banning users. The Client-Server API lets users update profile information, such as display names, avatars, and status messages. It manages presence information, indicating whether a user is online, offline, or idle, facilitating real-time communication.

The Client-Server API supports E2EE to ensure secure communications. It encrypts messages on the sender's device, allowing only the intended recipient to decrypt them, thereby guaranteeing privacy and security in all communications. Playing a key role in maintaining the integrity and efficiency of the Matrix network, the API standardizes how clients interact with homeservers and ensures compatibility across various client applications and devices. This compatibility enables users to select the client that best fits their needs without losing access to the network's full capabilities. The API further enhances

#friends: matrix.org

- {sender: "@bob:matrix.org", message: "hi"}
- {sender: "@alice:example.com", message: "morning"}
- {sender: "@carol:ufc.br", message: "hello"}

Fig. 2 The room #friends:matrix.org with simplified events

the user experience by supporting push notifications, media uploading, and content filtering features. These capabilities ensure that the Matrix ecosystem remains adaptable and can serve diverse user requirements, ranging from casual messaging to professional collaboration.

Servers and federation API

The synchronization process relies on the Server-Server API, also known as the Federation API. Servers, or homeservers, are the backbone of the Matrix protocol's decentralized architecture. Each homeserver is an independent server that stores and manages user data, including messages, media, user profiles, and room information. Homeservers are responsible for maintaining the state of the rooms where their users participate and ensuring that this state is synchronized with other homeservers across the network.

From a technical standpoint, a room consists of a sequence of JSON objects. Figure 2 illustrates a room with simplified events. The clients are responsible for correctly reading, parsing, and displaying these events.

Table 3 presents the essential events of the Server-Server API, including descriptions of each event's role in ensuring communication and synchronization between homeservers within the Matrix protocol. The events covered include Event Sending, Room Joining and Leaving, State Synchronization, and Event Authentication and Verification, highlighting their functions in maintaining consistency, integrity, and resilience across the decentralized network.

The Server-Server API defines the protocols and methods used for communication between homeservers, enabling them to exchange events, messages, and room state information. This ensures that all participating servers maintain a consistent view of the room state. When a user sends a message, it is stored on the sender's homeserver and then propagated to other homeservers via the Server-Server API. The process includes several key functions: Event Sending, Room Joining and Leaving, State Synchronization, and Event Authentication and Verification. In Event Sending, homeservers share events, such as messages or state updates, with other homeservers that have users participating in the same room, ensuring that all involved servers have the latest events. For Room Joining and Leaving, when a user joins or exits a room, their homeserver informs the other homeservers to ensure the room state is updated network-wide.

In the State Synchronization process, homeservers periodically synchronize room states to maintain consistency, comparing states, resolving conflicts, and agreeing on a common event history through a DAG model. Additionally, Event Authentication and Verification ensures the integrity and authenticity of events received from other homeservers. Each homeserver verifies events using cryptographic signatures, confirming that the data has not been altered during transmission and comes from legitimate sources. This combination of processes helps maintain a secure, synchronized, and consistent network across all homeservers.

The Server-Server API is essential for enabling federation, the core mechanism that allows different homeservers to communicate and collaborate in a decentralized manner. By facilitating seamless synchronization and communication between homeservers, the Server-Server API ensures that the Matrix network remains resilient, scalable, and consistent, even during network disruptions or server outages [1].

Figure 3 demonstrates three homeservers maintaining synchronization of their local copies of a room. In this configuration, each homeserver keeps its own version of the room's state, ensuring that all updates are propagated across the servers. When an event occurs in the room, it is shared among the homeservers, which then update their local states accordingly to remain in sync.

Homeservers operated by different individuals, organizations, or communities can coexist and communicate within the Matrix ecosystem. This setup maintains the integrity and continuity of user communications across a distributed network.

Identity servers and user discovery

Identity servers in the Matrix ecosystem play a crucial role in user discovery and connectivity by linking Matrix IDs (e.g., `@user:matrix.org`) to third-party identifiers, such as email addresses or phone numbers. This

Table 3 Essential events of the server-server API

Event	Description
Event Sending	Homeservers share events like messages and updates to keep room states consistent across servers.
Room Joining and Leaving	Homeservers update other servers when users join or leave rooms to maintain accurate room states.
State Synchronization	Homeservers periodically compare and synchronize room states using a Directed Acyclic Graph (DAG) model.
Event Authentication and Verification	Homeservers verify events from others using cryptographic signatures to ensure authenticity and integrity.

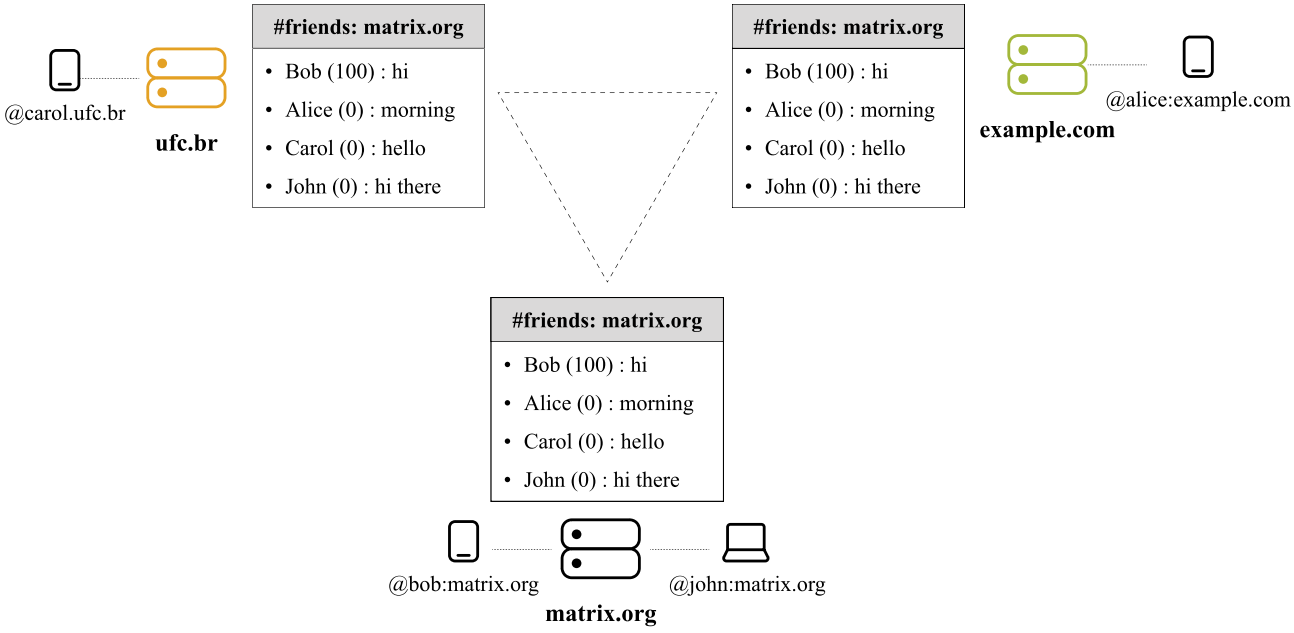


Fig. 3 Multiple homeservers maintain synchronization of their local room copies

Table 4 Main functions of identity servers and user discovery

Function	Description
User Discovery	Helps users find each other on the Matrix network by mapping third-party identifiers (e.g., email addresses, phone numbers) to Matrix IDs.
Contact Lookup	Allows users to check if their contacts are on Matrix by querying linked identifiers.
Privacy and Consent	Manages user consent for linking identifiers, ensuring privacy and compliance with data protection regulations.
Decentralization Support	Facilitates user discovery without relying on centralized services, supporting Matrix's decentralized nature.

functionality allows users to easily find each other on the Matrix network by searching with familiar identifiers, which the identity server then maps to the corresponding Matrix ID. This mapping service is especially valuable for onboarding new users who might not be familiar with Matrix ID formats or when integrating existing contact lists from other platforms [1]. The use of identity servers is particularly useful for simplifying user interactions and enhancing connectivity; however, it also raises important privacy considerations. To address these, Matrix allows multiple identity servers to operate independently, giving users the choice of which server to trust and ensuring they have control over their data. This decentralized approach helps safeguard user privacy and aligns with Matrix's broader goals of decentralization and resilience against censorship. These functions collectively contribute to a robust, privacy-respecting, and user-friendly environment, aligning with Matrix's commitment to open and decentralized communication. By facilitating seamless user discovery, efficient contact lookup, stringent privacy management, and support for decentralized operations, identity servers significantly enhance the functionality and appeal of the Matrix protocol. Table 4 describes these functions in detail, highlighting their importance in maintaining the integrity and usability of the Matrix network.

Event-based communication model

Matrix operates on an event-based communication model, where every action within the network is captured as an event. These events cover a wide range of actions, including sending messages, joining rooms, updating profiles, or modifying room settings. Each event is carefully timestamped and stored in chronological order, ensuring that all participants in a room observe the same sequence of messages and actions, regardless of the homeserver they are connected to. This structure guarantees a consistent and synchronized experience across the network, allowing users to follow the conversation's flow seamlessly, no matter their location or server connection.

In Matrix, events are represented as immutable objects within a DAG. The DAG structure enables each event to reference one or more preceding events, creating a chain

that establishes the order of occurrences. This model allows homeservers to maintain a coherent view of the room's state without requiring a central authority to manage the sequence of actions. As a result, the Matrix network ensures data integrity and coherence across its distributed architecture, supporting a decentralized and resilient system for communication. The event-based DAG model enhances reliability and transparency, enabling efficient collaboration and secure interactions throughout the network.

Decentralization

Decentralization is a core principle of the Matrix protocol. Unlike traditional centralized messaging platforms, where all data is controlled by a single entity (e.g., a company that owns the messaging service), Matrix allows anyone to run their own homeserver. This means that users have control over their data, and the network is not dependent on any single point of failure [1]. Matrix's decentralized architecture enhances resilience against outages and censorship. If one homeserver goes offline, the data stored on other homeservers remains intact, and communication can continue. This makes Matrix particularly appealing in environments where censorship is a concern, as it is much harder to suppress information on a decentralized network [1].

Federation plays a crucial role in decentralization by allowing homeservers to communicate and synchronize data with each other. Each homeserver in the network is autonomous, yet they cooperate to maintain a consistent and coherent view of the communication across the network. This cooperation ensures that users on different homeservers can interact as if they were on the same server [1]. Furthermore, Matrix supports peer-to-peer (P2P) communication, where users can communicate directly without the need for a homeserver. This is particularly useful in scenarios where infrastructure is limited or where users want to avoid relying on third-party servers altogether [1].

Security

Security is one of the main pillars of the Matrix protocol, with E2EE being a critical feature. E2EE ensures that messages are encrypted locally before transmission. Only the recipient, holding the correct decryption key, can access the original content. Even if intercepted during transfer, the content of the communication remains secure and unreadable by unauthorized parties [1]. Table 5 summarizes the cryptographic functions employed by the Matrix protocol, detailing its core mechanisms. Each key type – Ed25519, Curve25519, Olm, and Megolm – plays a role in securing and maintaining the integrity of communications within the Matrix network.

Matrix uses the Olm and Megolm cryptographic ratchets to implement E2EE. Olm is used for one-to-one communication, while Megolm is designed for group communication. These ratchets ensure forward secrecy, meaning that even if one encryption key is compromised, past and future communications remain secure [1]. The Olm and Megolm protocols rely on cryptographic primitives like Ed25519 and Curve25519 to achieve their security properties. Ed25519 is used for digital signatures, providing a secure method to authenticate the sender of a message. Curve25519, on the other hand, is used for key exchange, allowing two parties to securely establish a shared secret over an insecure channel [17].

In addition to E2EE, Matrix employs other security measures, such as device verification and cross-signing. Device verification allows users to confirm the authenticity of each other's devices by comparing unique device fingerprints. This process ensures that users communicate with the correct devices, preventing man-in-the-middle (MITM) attacks. Cross-signing enables users to verify a new device once and have that verification apply to all their other devices automatically, streamlining the security process across multiple devices [1]. Matrix also prioritizes transparency and auditability. The protocol's decentralized nature ensures that no single entity can alter or delete data without consensus, and all actions

Table 5 Functions of keys in the Matrix protocol

Key Type	Function
Ed25519	Used for digital signatures.
	Ensures the authenticity of the sender.
	Provides message integrity by verifying that the content has not been tampered with.
Curve25519	Utilized for key exchange.
	Allows two devices to establish a shared secret securely.
	Fundamental for setting up encrypted communication channels between devices.
Olm	Implements a ratcheting protocol for E2EE in one-to-one communications.
	Combines Curve25519 for key exchanges and Ed25519 for signatures.
	Provides forward secrecy, post-compromised security, and deniability.
Megolm	An adaptation of the Olm protocol designed for group communications.
	Uses a single ratchet shared among all group members.
	Allows efficient encryption while maintaining forward secrecy and message integrity.

are recorded as events, providing a clear audit trail. This auditability is crucial for maintaining trust in a decentralized system, as it allows users to verify that their data has not been tampered with [1].

Security boundaries in federated environments

Matrix allows users to operate their own homeservers, which increases autonomy over infrastructure and key management. However, this does not imply full data ownership. Due to federation, messages are often replicated to other homeservers participating in the same room. This replication reduces the level of control an individual user has over their data, particularly in public or multi-party conversations [18].

Forward secrecy in Matrix ensures that past messages remain confidential even if current or future encryption keys are compromised. However, this mechanism does not protect future communication. It prevents retrospective decryption but not prediction or interception of messages yet to be sent [1].

Matrix does not offer strong metadata protection by default. Elements such as message timestamps, room membership, and homeserver domains can be inferred through traffic analysis [19]. While message contents remain encrypted, metadata exposure remains a challenge for privacy-sensitive use cases. On the client side, protection mechanisms vary across implementations. Typically, encrypted messages are stored locally, and key handling is done using secure enclaves or encrypted key stores when available.

Security mechanisms and future considerations

Table 6 summarizes the main cryptographic mechanisms used in the Matrix protocol. Each algorithm supports a specific security property, such as confidentiality, integrity, or authentication. This classification provides a clearer view of how security goals are achieved across different layers of the protocol.

Many of these mechanisms were introduced or improved in specific protocol versions. End-to-end encryption (E2EE) became a standard feature in version 1.0. Improvements in device verification and key management appeared in later versions, such as 1.1 and 1.3. The most recent releases (1.10 and 1.11) introduced updates to encryption handling and protections against abuse.

Matrix currently uses Ed25519 and Curve25519 for signatures and key exchanges. These elliptic curves are expected to become deprecated after 2035, due to their vulnerability to quantum computing attacks [20]. Although the Matrix specification does not yet define a post-quantum transition, discussions in the open-source community have explored alternatives such as CRYSTALS-Dilithium and Kyber, which are among the algorithms selected by NIST for standardization [21].

Future versions of the protocol may incorporate hybrid or post-quantum schemes to mitigate long-term risks. However, challenges remain in balancing security, performance, and compatibility across a wide range of client implementations.

Interoperability

Interoperability is a fundamental goal of the Matrix protocol, enabling it to bridge communication silos and allow users on different platforms to interact seamlessly. Matrix was designed to connect disparate communication systems, making it possible for users on Matrix to communicate with users on other platforms like Slack, Discord, IRC, or even traditional SMS [1].

Matrix achieves interoperability through the use of bridges. Bridges are connectors (Fig. 4) that translate data and protocols between Matrix and other communication systems. For example, a Matrix-Slack bridge would allow users in a Matrix room to communicate with users in your network as if they were in the same chat room. This bridging capability is one of Matrix's most powerful features, as it creates a unified communication experience across different platforms [1].

In addition to bridging, Matrix's open standard and API-driven architecture make it easy for developers to build custom integrations and applications on top of the protocol. This extensibility has led to developing a wide range of clients, bots, and services that enhance the Matrix ecosystem. Its support for other standards and protocols also reflects Matrix's commitment to openness and interoperability. For example, Matrix can integrate with WebRTC for real-time voice and video communication, and it supports the integration of decentralized identity solutions, making it a versatile platform for modern communication needs [1].

Table 6 Cryptographic mechanisms in Matrix and their purposes

Purpose	Mechanism	Details
Confidentiality	Olm / Megolm	End-to-end encryption of messages
Key Exchange	Curve25519	Derives shared secrets between devices
Integrity / Authentication	Ed25519 (digital signatures)	Event signing and verification
Forward Secrecy	Olm (1:1)	Per-message ratcheting
Group Messaging	Megolm	Shared symmetric ratchet for groups

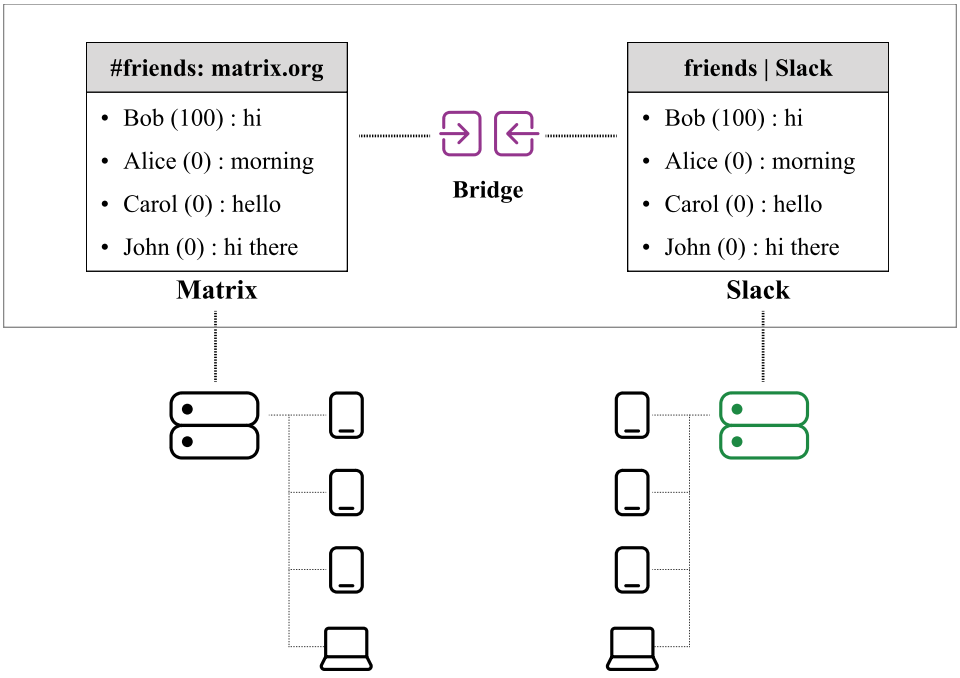


Fig. 4 A room bridged between matrix.org and slack.com

Table 7 Typical use cases of the Matrix protocol

Use Case	Description
Instant Messaging	Secure, encrypted text messaging between users.
Private Group Chat	Secure spaces for group messaging and file sharing.
Public Rooms	Open forums for discussions, support, and events.
VoIP Signaling	Secure voice and video calls using Matrix's messaging mechanism.
Internet of Things (IoT) Communication	Secure data exchange between IoT devices in a decentralized manner.
Bridging Existing Communication Silos	Connecting different platforms like Slack, Discord, Telegram, and IRC for seamless communication.

Use and applications of the matrix protocol

The Matrix protocol was designed to provide a flexible and secure platform for decentralized communication. Since its launch, it has been adopted in various use cases, from instant messaging and group chats to online communities and integration with other communication platforms. In this section, we will discuss some of the typical use cases of Matrix, examples of platforms and services that utilize the protocol, and the benefits and challenges of adopting Matrix.

Typical use cases of Matrix

Matrix's use cases are broad and diverse, reflecting its flexibility and ability to adapt to different communication needs. Table 7 outlines the typical use cases of the Matrix protocol, showcasing its versatility in supporting various communication needs. It includes applications ranging from instant messaging and private group chats to public rooms for large-scale interactions. Additionally, the protocol facilitates secure voice and video calls. It enables communication between IoT devices and bridges different communication platforms. These capabilities highlight Matrix's adaptability and strength in fostering decentralized, safe, and interconnected communication environments.

Matrix serves as an instant messaging platform, allowing users to exchange text messages quickly and securely through E2EE, ensuring that only participants can access the content of conversations. Beyond individual messaging, Matrix enables the creation of private group chats where users can share messages and files in a protected environment, making it ideal for work teams, families, or any group needing a secure space for communication. In addition to private chats, Matrix supports public rooms where any user can join and participate in discussions. These rooms are commonly used for forums, technical support, or live events, providing an organized space for large groups to interact. Another notable feature is Matrix's use for VoIP signaling, which enables secure and decentralized voice and video calls. The same event mechanism used for messaging also facilitates calls, ensuring encryption and privacy, making Matrix a strong option for real-time multimedia communication.

Moreover, Matrix's capabilities extend to communication between IoT devices, allowing them to exchange data securely without relying on a central server. This decentralized approach is crucial for scenarios where privacy and data security are paramount. Additionally, Matrix excels at bridging communication silos by connecting users across different platforms like Slack, Discord, Telegram, and IRC networks. By acting as a bridge, Matrix allows seamless communication across previously isolated systems, making it a versatile solution for both personal and professional use. The flexibility in connecting various platforms and supporting secure communication makes Matrix a comprehensive tool for diverse communication needs.

Communities in Matrix

The concept of communities in Matrix stands out as one of the protocol's most powerful and flexible features. Communities allow users to create and share spaces that can group multiple rooms, streamlining the organization and moderation of large-scale discussions. In Matrix, spaces serve as collections of rooms that can be arranged into hierarchies, enabling more efficient management of interactions. A space can represent various entities, such as a community, project team, or interest group. Space creators are given the tools to add rooms, set permissions, and invite members, establishing a collaborative and structured environment where communication can flourish.

Space administrators play a key role in maintaining order within these communities. They have the ability to add new rooms, assign moderators, and set rules for participation. Basic moderation tools allow administrators to ban users, set discussion topics, and control access to rooms, ensuring the community remains organized and secure. This level of control enables communities to scale effectively while maintaining a safe and well-structured environment for participants to interact. Through these capabilities, Matrix provides an adaptable solution for creating, managing, and moderating large-scale conversations across various interest groups and projects.

Matrix ecosystem

The Matrix ecosystem is vast and diverse, with many clients, servers, and bridges extending its capabilities. Figure 5 illustrates the Matrix ecosystem, highlighting its key components and their interactions. Label 1 represents the users or clients who interact with the homeservers. Label 2 identifies the homeservers, which are responsible for managing communication and data storage for clients, while also federating with other homeservers to maintain a decentralized and interconnected network. Label 3 points to identity servers, which handle user identification and authentication within the Matrix network. Label 4 indicates service applications, such as bridges, which facilitate connections between Matrix and other communication platforms. Finally, Label 5 represents communication service providers that can integrate with and access the Matrix network, broadening

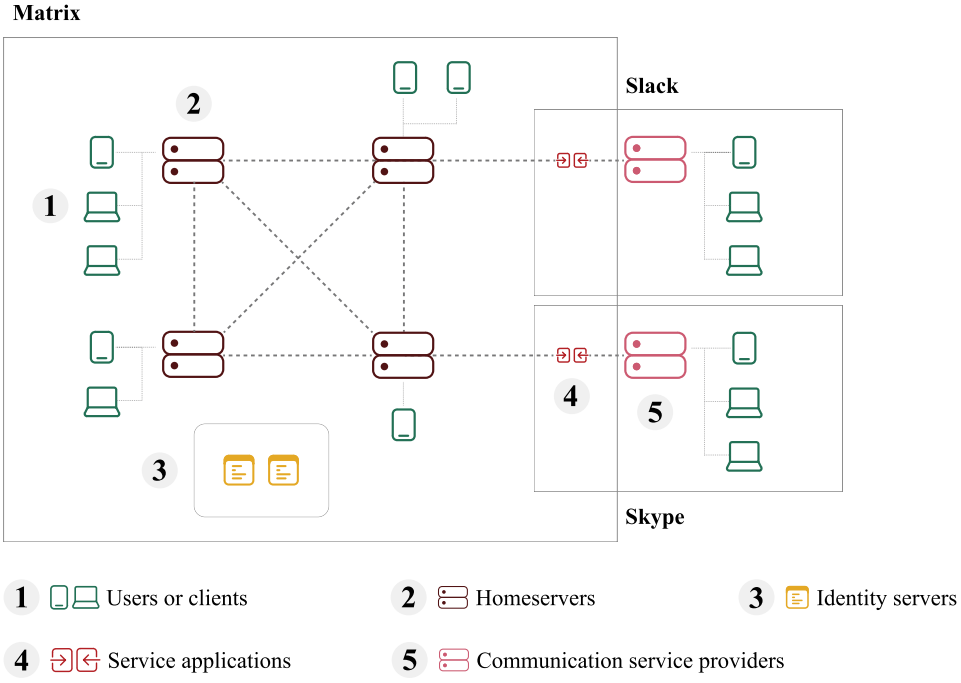


Fig. 5 The Matrix ecosystem with all its components, including clients and servers

Table 8 Key components of the Matrix ecosystem

Component	Description
Clients	Element, Element X, FluffyChat, and Hydrogen.
Servers	Synapse, Dendrite, and Conduit.
Bridges	Slack, Discord, and IRC.

Table 9 Related work comparison

Ref.no.	Year	Type of Research	Focus	Considered Applications	Objective
[26]	2021	Survey	Security	Signal, WhatsApp, Wire, Viber, Riot, Telegram	Compare applications
	2023	Survey	Usability	Mattermost, Matrix, Element, Cinny, Gitter, Zulip, Twake, Rocket.Chat, Corteza, Revolt	Compare applications
[27]	2023	Survey	Interoperability	Signal, WhatsApp, Matrix	How to integrate applications
[28]	2023	Survey	Security	Element, WhatsApp	Compare applications
[4]	2023	Survey	Security	Element, Facebook Messenger, FaceTime, Signal, Telegram, WhatsApp, and others	Compare applications
This paper	2024	Sistematic Mapping Study	Relationship of the paper with Matrix, area, main contribution, explored Matrix feature	Totally focused on Matrix	Analyze the Matrix

the scope of connectivity and interoperability within the ecosystem. This comprehensive setup underlines Matrix's decentralized and federated nature, allowing seamless and secure communication across different services and platforms.

Table 8 presents the key components of the Matrix ecosystem, highlighting the variety of clients, servers, and bridges that contribute to its versatility and interoperability. The Matrix protocol supports diverse clients, from the widely recognized Element to other alternatives like FluffyChat [15], Hydrogen [6], and SchildiChat [22], each offering unique user experiences. Matrix servers such as Synapse [23], Dendrite [24], and Conduit [25] also form the network's backbone, catering to different performance and feature requirements. Moreover, Matrix's ability to bridge connections with platforms like Slack, Discord, Telegram, and IRC underscores its commitment to seamless cross-platform communication.

Related work

In the rapidly evolving field of communication and messaging systems, a substantial body of research has emerged, addressing various aspects such as security, usability, and interoperability. This section reviews key studies that have shaped our understanding of these topics, as summarized in Table 9. The table provides a comparative overview of significant papers that have performed some type of literature review in this topic. The comparison details their year of publication, type of research, focus, considered applications and objective. By examining these contributions, we highlight the progression of research and identify the specific niche our study addresses. Our research stands out by offering a detailed mapping study of the Matrix protocol – a topic that has not been extensively explored in prior work. Following, we discuss each table's column.

Year. Research in communication and messaging systems has seen considerable growth and diversification in recent years. Foundational studies began as early as 2021, laying the groundwork for deeper exploration into various aspects of these technologies. The year 2023 was particularly noteworthy, with a surge of significant research contributions, such as those by Andriessen et al. and Blessing et al. [27], reflecting heightened interest and activity in the field. Our study, developed in 2024, builds upon this momentum by delivering a focused analysis of the Matrix protocol. The Matrix protocol is increasingly recognized for its potential to enhance interoperability and security in decentralized communication systems. Our paper highlights the ongoing evolution and refinement of this protocol, offering fresh insights and contributing to its growing importance within the broader landscape of messaging systems.

Type of Research. The body of research in communication and messaging systems includes various methodological approaches, each serving distinct purposes. Surveys, as seen in papers like Johansen et al. [26] and Blessing et al. [27], offer comprehensive overviews of existing technologies and practices, often focusing on comparisons across multiple platforms. These studies are valuable for understanding general trends and benchmarks within the field. However, they typically provide limited, not systematic, high-level insights rather than detailed analyses of specific technologies. In contrast,

our study utilizes a systematic mapping study approach [29]. This methodology is crucial for thoroughly examining a particular technology's features, contributions, and its role within the broader ecosystem. By systematically mapping the Matrix protocol, our research fills a critical gap by providing a detailed, structured analysis that highlights its unique aspects and contributions, which are often overlooked in more general surveys. This approach allows us to offer deeper insights into the Matrix protocol.

Focus. Previous research on communication and messaging systems has predominantly concentrated on critical areas such as security, usability, and interoperability. For instance, studies by Johansen et al. [26] and Blessing et al. [28] focus heavily on security, highlighting the ongoing concern with safeguarding communication data. Usability is the primary focus of Andriessen et al. , which addresses how user-friendly different systems are. Additionally, Blessing et al. [27] concentrate on interoperability, examining how well various systems integrate with one another. In contrast, our research concentrates on the Matrix protocol. This targeted analysis offers new insights into the Matrix protocol's role and performance that are not as thoroughly examined in the existing literature.

Considered Applications. The applications examined in the literature vary widely, including Signal, WhatsApp, Matrix, and others. Research such as Andriessen et al. and Alatawi et al. [4] covers a broad range of platforms, while Blessing et al. [27] focuses on fewer applications. Our study stands out by concentrating exclusively on the Matrix protocol, offering a comprehensive analysis of its capabilities and contributions. This targeted examination provides a detailed understanding of Matrix that contrasts with the broader application scopes of other studies.

Objective. The objectives outlined in the table reflect a range of approaches to understanding and evaluating communication and messaging systems. Most prior research, including the surveys by Johansen et al. [26], Andriessen et al. , and Blessing et al. [27], aim to compare various applications across different criteria, such as security, usability, and interoperability. These studies generally focus on providing a broad overview and benchmarking of multiple systems to understand their relative strengths and weaknesses. In contrast, our research, takes a distinctive approach by conducting a systematic mapping study specifically focused on the Matrix protocol. The objective of our study is to offer an in-depth analysis of the Matrix protocol, exploring its key features, contributions, and the relationships it forms within the messaging landscape. By concentrating on this single protocol, our paper seeks to provide a detailed and comprehensive understanding of Matrix.

How has the Matrix protocol been used in academic research?

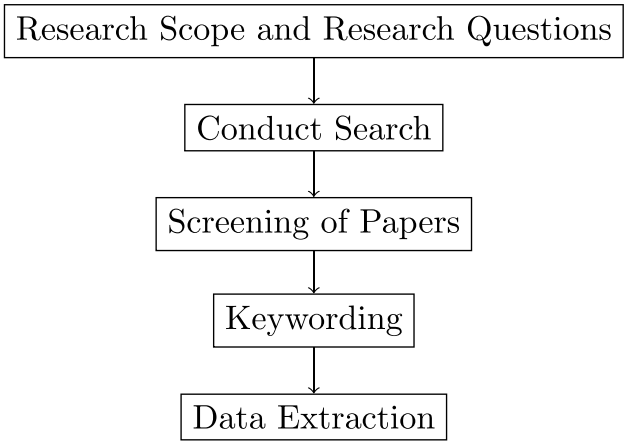


Fig. 6 Steps of the mapping study conducted in this research

Methodology

This section outlines the methodology employed in this work. We utilized a systematic mapping study to explore the topic of interest. According to Kitchenham et al. [29], a systematic mapping study involves "a broad review of primary studies in a specific topic area that aims to identify what evidence is available on the topic". Systematic mapping studies provide several benefits. They offer a comprehensive overview of a research area, highlight gaps in the existing research, and support evidence-based decision-making. By systematically reviewing the literature, researchers can assess the current state of knowledge, pinpoint areas needing further investigation, and guide future research efforts. These studies help to synthesize large amounts of information, reveal trends and patterns, and identify inconsistencies across different studies. They are valuable resources for policy-makers, practitioners, and researchers. Moreover, systematic mapping studies can reduce research duplication, improve resource allocation, and foster collaboration among researchers in related fields. We adopted the systematic mapping study process as outlined by Peterson et al. [30], which is illustrated in Fig. 6 and detailed in the subsequent subsections.

Research Scope and Research Questions

Understanding the cutting-edge methods for evaluating the Matrix protocol's performance is essential in today's digital landscape, where the adoption of decentralized communication systems is rapidly growing. As organizations increasingly implement the Matrix protocol to facilitate secure and scalable communication, evaluating its performance becomes critical. The swift advancements in decentralized technologies introduce complexities that necessitate novel evaluation methods and tools. By

Table 10 List of research questions

Question IDs	Description
RQ01	What is the relationship between the paper and the Matrix protocol?
RQ02	What is the main research area of the paper?
RQ03	What is the main contribution of the paper?
RQ04	What is the explored Matrix protocol feature in the paper?

examining the current state-of-the-art areas, contributions, and Matrix features, we can gain insights into its performance and identify areas needing improvement. This exploration will help uncover innovative evaluation strategies and highlight gaps in the existing literature, providing a solid foundation for future research aimed at optimizing the Matrix protocol's performance and functionality. Based on these considerations, this paper seeks to answer the following question:

We can delve into more specific inquiries stemming from the general question. Our focus is on addressing four key questions outlined in the Table 10.

RQ01. Understanding the *relationship between research papers and the Matrix protocol* is essential for evaluating how the protocol integrates into the broader academic and practical discussions on decentralized communication. The Matrix protocol, known for its robust framework supporting decentralized and secure messaging, has been increasingly influential across various research domains, including cybersecurity, distributed systems, and real-time communication. By examining this relationship, we can understand how scholars and practitioners are integrating the Matrix protocol into their research. This may involve focusing on the protocol itself or using it as a component in broader studies of decentralized technologies. This analysis not only helps to trace the protocol's impact on the field but also reveals how it influences and is shaped by ongoing research efforts, offering a clearer understanding of its role in advancing communication technologies.

RQ02. Identifying the primary *research area of papers* that explore the Matrix protocol is crucial for contextualizing its application and relevance. The Matrix protocol intersects with several research domains, such as cybersecurity, real-time communication, and network architecture. By determining the main research area, we can better understand how the Matrix protocol is applied within these fields and assess its contributions to solving specific problems or advancing knowledge in these domains. This clarity helps in recognizing trends and emerging challenges related to the protocol and provides direction for future research efforts aimed at leveraging or enhancing its capabilities.

RQ03. Evaluating the *main contribution of the papers* that investigates the Matrix protocol highlight the advancements and innovations it brings to the field. Contributions can range from introducing new evaluation methodologies for the protocol, proposing enhancements to its existing features, or providing novel insights into its practical applications and performance. Understanding these contributions allows us to appreciate how the paper advances the development and deployment of the Matrix protocol. It also helps in identifying specific areas where the protocol is being improved or expanded, which is crucial for addressing gaps in current research and guiding future investigations. This knowledge is vital for recognizing the paper's impact on both theoretical and practical aspects of decentralized communication systems.

RQ04. Investigating the *specific feature of the Matrix protocol* that a paper explores provides a detailed understanding of the focus and scope of the research. The Matrix protocol encompasses a variety of features such as E2EE for secure messaging, interoperability with other communication systems, and decentralized data storage. By identifying which feature is the primary focus of the paper, we can assess how it addresses specific challenges or opportunities related to that feature. This insight helps in understanding the current priorities in research on the Matrix protocol and highlights areas where further development or investigation may be needed. It also provides a deeper appreciation of how different aspects of the protocol are being examined and optimized, contributing to the overall evolution of decentralized communication technologies.

Conduct search (For Primary Studies)

We conducted a search across the electronic sources listed in Table 11, focusing on publications that cited terms related to the Matrix protocol, such as "Matrix protocol," "matrix.org," and other related variations. We chose to include "matrix.org" in our search because it is the primary repository for the Matrix protocol's documentation. Our search strategy involved examining the full text of these publications, which enabled us to identify papers specifically addressing the Matrix protocol, including those discussing its key features such as security, decentralization, and chat mechanisms.

Screening of papers (For inclusion and exclusion)

Inclusion and exclusion criteria are indispensable elements of mapping studies, serving as a navigational compass that guides researchers through the vast sea of literature. These criteria ensure precision in research

Table 11 Search strings

Search Engines	Quantity	Search String
IEEE Xplore	16	"matrix.org" OR "matrix protocol" OR "protocol matrix"
ACM Digital Library	20	[All: "matrix protocol"] OR [All: "protocol matrix"] OR [All: "matrix.org"]
Google Scholar	12	Search for "matrix.org", "matrix protocol", "protocol matrix" then click on Sort by date - Abstracts
ScienceDirect	82	Title, abstract, keywords, References: "matrix.org" OR "matrix protocol" OR "protocol matrix"
SpringerLink	26	"matrix.org" OR "matrix protocol" OR "protocol matrix"
TOTAL	**156**	

Table 12 Inclusion and exclusion criteria

Inclusion	Exclusion
• The abstract or title explicitly mentions the Matrix protocol.	• The paper does not conduct experiments, analyses, comparisons, or correlations using the protocol Matrix.
• Perform a practical experiment with the Matrix protocol.	• Not written in English.
• Performs formal analysis on the Matrix protocol.	
• Only Mentions through comparison or correlation.	

Table 13 Primary studies detailed as references

Ref.no.	Title
[12]	"A Glimpse of the Matrix: Scalability Issues of a New Message-Oriented Data Synchronization Middleware", Jacob, F. et al., 2019, Middleware'19.
[3]	"Matrix Decomposition: Analysis of an Access Control Approach on Transaction-Based DAGs without Finality", Jacob, F. et al., 2020, SACMAT'20.
[11]	"Analysis of the Matrix Event Graph Replicated Data Type", Jacob, F. et al., 2021, IEEE Access.
[31]	"Help, My Signal Has Bad Device!", Wichelmann, J. et al., 2021, DIMVA 2021.
[9]	"Key Agreement for Decentralized Secure Group Messaging", Weidner, M. et al., 2021, CCS 2021.
[32]	"Pak e-Mail: Authentication and Key Management in Decentralized Secure Email", Vazquez Sandoval, I. et al., 2021.
[2]	"Forensic Analysis of Matrix Protocol and Riot.im Application", Schipper, G.C. et al., 2021, Forensic Science Int.
[33]	"Extracting Entities from Production Chats", Müller, M., 2021, Procedia CIRP.
[34]	"Safe and Private Forward-Trading Platform for Microgrids", Eisele, S. et al., 2021.
[19]	"DRIFT: E2EE Spatial Feature Sharing & Messaging", Brunila, M. et al., 2022, SIGSPATIAL.
[35]	"teacher.js: Low-bandwidth Tool for Remote Teaching", Brodbeck, F. et al., 2022, IEEE ECEI.
[36]	"Towards Portable Identities in the Matrix Protocol", Ihle, C. et al., 2022, ICDCSW.
[37]	"Impact of WhatsApp's 2021 Privacy Policy", Griggio, C.F. et al., 2022, CHI 2022.
[38]	"Threat Models of E2EE Messaging Apps", Chowdhury, P.D. et al., 2023, arXiv.
[39]	"Fork-Resilient Group Key Agreement", Alwen, J. et al., 2023, CRYPTO 2023.
[40]	"Where on Earth is the Spatial Name System?", Gibb, R. et al., 2023, HotNets'23.
[14]	"Demystifying Decentralized Matrix Communication", Li, H. et al., 2023, ICPADS.
[13]	"Cryptographic Vulnerabilities in Matrix", Albrecht, M.R. et al., 2023, IEEE S&P.
[41]	"Comparse: Secure Formats for Cryptographic Protocols", Wallez, T. et al., 2023, CCS 2023.
[42]	"CrossChat: Messaging Across Different Apps", Cui, M. et al., 2023, ACM SAC.
[43]	"Matrix-Mesh Communication Protocol", Rahman, M. et al., 2024, Dew Computing.

scope by delineating the boundaries of relevance, allowing scholars to focus exclusively on literature that directly addresses their research questions and objectives. They minimize bias by providing transparent guidelines for selection and optimize resource utilization by efficiently allocating time and effort to synthesizing high-quality evidence. Moreover, adherence to these criteria improves the reproducibility and transparency of mapping studies, enabling other researchers to validate findings and build upon existing knowledge. In essence, inclusion and exclusion criteria are the cornerstone of methodological rigor in mapping studies. They facilitate the synthesis of meaningful insights that contribute to the advancement of scholarly discourse and understanding in various fields. Table 12 presents the inclusion and exclusion criteria for this work. Table 13 shows the 21 selected studies.

Table 14 Unified keywords

Classification	Keywords
Relationship Between the Article and The Matrix	Uses the Matrix Protocol in Practice; Only Mentions.
Main Area of the Paper	Communication and Networks; Security and Privacy; Forensic Analysis; Natural Language Processing (NLP); Educational Technology; Human-Computer Interaction (HCI).
Main Contribution	Matrix Protocol Analysis; Security and Privacy Approach; Protocol Development; Tool Development.
Explored Matrix Feature	Group Communication; Security Issues; Matrix Architecture.

Table 15 Categorization of each paper

Ref.no.	Relationship between the paper and the Matrix	Main area of the paper	Main contribution	Explored Matrix feature
[12]	Uses the Matrix Protocol in Practice	Communication and Networks	Matrix Protocol Analysis	Group Communication
[3]	Uses the Matrix Protocol in Practice	Communication and Networks and Security and Privacy	Matrix Protocol Analysis / Security and privacy Approach	Matrix Architecture
[11]	Uses the Matrix Protocol in Practice	Communication and Networks and Security and Privacy	Matrix Protocol Analysis / Security and privacy Approach	Matrix Architecture
[31]	Only Mentions	Security and Privacy	Security and privacy Approach	Security Issues
[9]	Uses the Matrix Protocol in Practice	Security and Privacy	Protocol development	Security Issues
[32]	Only Mentions	Security and Privacy	Protocol development	Group Communication
[2]	Uses the Matrix Protocol in Practice	Forensic analysis	Matrix Protocol Analysis	Group Communication
[33]	Uses the Matrix Protocol in Practice	Natural Language Processing	Tool development	Group Communication
[34]	Only Mentions	Security and Privacy	Tool development	Security Issues
[19]	Uses the Matrix Protocol in Practice	Security and Privacy	Tool development	Group Communication and Security Issues
[35]	Uses the Matrix Protocol in Practice	Educational Technology	Tool development	Group Communication
[36]	Uses the Matrix Protocol in Practice	Security and Privacy	Security and privacy Approach	Group Communication
[37]	Only Mentions	HCI	Security and privacy Approach	Group Communication
[38]	Uses the Matrix Protocol in Practice	Security and Privacy	Security and privacy Approach	Security Issues
[39]	Only Mentions	Communication and Networks	Protocol development	Security Issues
[40]	Only Mentions	Communication and Networks	Protocol development	Group Communication
[14]	Uses the Matrix Protocol in Practice	Security and Privacy	Matrix Protocol Analysis Security / privacy Approach	Matrix Architecture
[13]	Uses the Matrix Protocol in Practice	Security and Privacy	Matrix Protocol Analysis / Security and privacy Approach	Matrix Architecture
[41]	Only Mentions	Security and Privacy	Tool development	Security Issues
[42]	Only Mentions	Communication and Networks / HCI	Tool development	Group Communication
[43]	Uses the Matrix Protocol in Practice	Communication and Networks	Protocol development	Matrix Architecture

Keywording

We have found 15 expressions, which were grouped and presented in Table 14. The expressions were unified when they appeared with similar meanings, as suggested by Peterson et al. [30]. The analysis categorized papers based on their relationship to the Matrix protocol, distinguishing between those that use the protocol in practice and those that merely mention it. We also classified the main research areas covered in the literature, which include Communication and Networks, Security and Privacy, Forensic Analysis, Natural Language Processing (NLP), Educational Technology, and Human-Computer Interaction (HCI). Furthermore, the main contributions of the papers were identified as Matrix Protocol Analysis, Security and Privacy Approaches, Protocol Development, and Tool Development. Finally, we explored specific Matrix features addressed in the research, such as Group Communication, Security Issues, and Matrix Architecture. This classification allowed us to comprehensively understand the scope and impact of the Matrix protocol within various research domains and its specific applications and contributions.

Data extraction

In this step, we associate the keywords to each paper, producing the results listed in Table 15. This table allows the reader to access the specific paper that addresses the desired characteristics.

Table 15 synthesizes the main findings of this mapping study. It classifies each primary study along four dimensions: its relationship with the Matrix protocol, the main research area, the type of contribution, and the specific

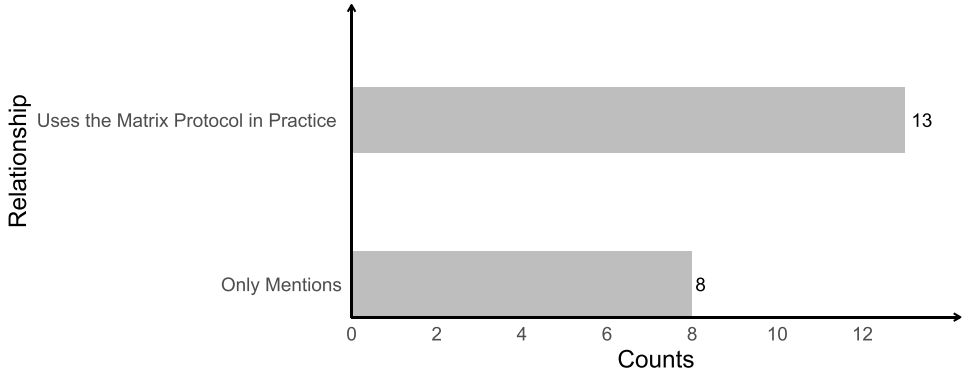


Fig. 7 Primary studies quantified by relationship

protocol features addressed. This structured classification enables a clearer understanding of how Matrix has been used and studied across different domains.

The table helps identify trends, such as the dominance of security-focused research, and reveals gaps in underexplored areas like usability and interoperability. It also supports comparisons across studies, showing overlaps and divergences in methodology and focus. Due to its integrative value, we consider Table 15 a significant contribution of this work, as it provides a reusable reference for researchers and practitioners working on decentralized communication systems.

Research answers

Now, we will analyze each of the research questions one by one. First, we will explain the meaning of each keyword used to categorize the papers. Next, the information will be presented in graphical format.

RQ01 - Relationship with Matrix

Figure 7 shows the relationship between the papers and the Matrix protocol. The papers were classified as **Uses the Matrix Protocol in Practice** and **Only Mentions**.

Thirteen papers have **used the Matrix protocol in practice**. These papers use the Matrix protocol in experiments based on code implementation or through formal analysis. We believe that, given that Matrix is a set of open APIs for federated and open communication of Instant Messaging (IM), VoIP, and IoT, it is natural for researchers to use it in practical experiments. Rahman et al. [43], for example, have developed a prototype of the mesh-matrix communication protocol for Local Area Networks (LANs), Personal Area Networks (PANs), Bluetooth, and Near Field Communication (NFC) networks was developed. Implementing the mesh-matrix communication protocol prototype extends the Matrix Protocol functionality to support networks such as LANs, PANs, Bluetooth, and NFC, enhancing its versatility and accessibility across different communication mediums. Brodbeck et al. [35] proposed an educational tool utilizing the Chat App of the Matrix protocol. This educational tool aims to enhance the learning experience by integrating real-time communication features with collaborative tools. Brunila et al. [19] developed the DRIFT, a messaging app that follows a unique communication approach, maintaining time sequence and spatial organization. It utilizes maps to show the message locations. Other papers focus on formal and asymptotic analyses of data replication mechanisms and structuring a Matrix network [3, 11, 12].

Eight papers were assigned the **Only Mentions** category. These papers cover various topics and reference the Matrix protocol. In this way, the Matrix protocol was used as an example of a technology that supported or refuted a particular research feature. Therefore, these mentions can range from briefly comparing technologies to valuable insights. Finally, we believe these papers have contributed significantly to the scientific dissemination of the ecosystem created by the Matrix protocol. Cui et al. [42] have introduced CrossChat enables interaction between users operating on different mobile messaging platforms. It implements a gateway on the device that mediates communication and leverages interface-level interaction techniques to route messages across applications in real time. The method used in CrossChat closely resembles the function of Matrix bridges, which connect different messaging networks, enabling communication across disparate ecosystems. Wichelmann et al. [31] explores the security aspects of the Signal protocol. The Matrix protocol was used to highlight device-specific security code extensions.

RQ02 - Main area of study

Figure 8 shows the papers according to their main area of study. The topic of **Security and Privacy** was addressed in twelve papers, which discuss a wide range of measures to protect user data and ensure secure communication channels. These measures include encryption, which involves encoding data to prevent unauthorized access, authentication processes to verify the identity

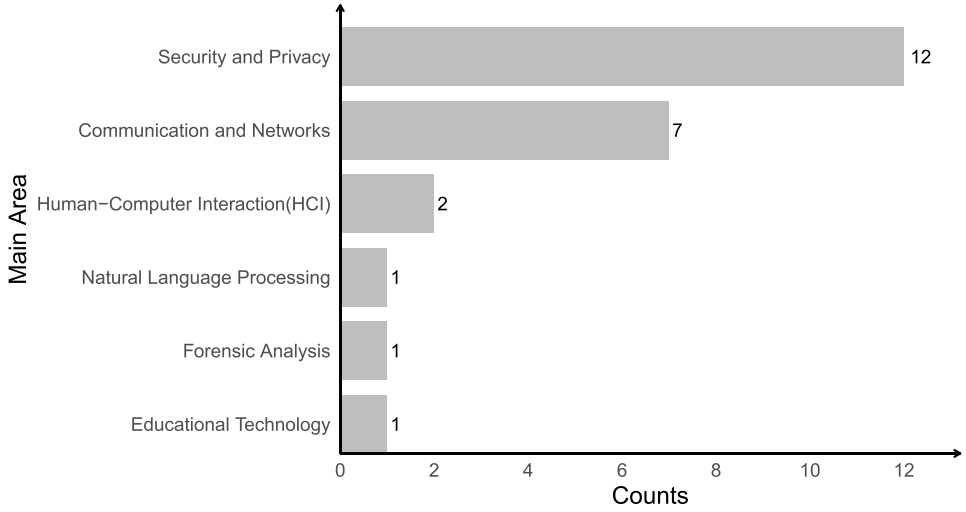


Fig. 8 Primary studies, quantified by main area

of users, and access control mechanisms to manage and restrict user permissions. Most papers addresses **Security and Privacy** category. We believe that the Matrix communication model naturally raises security concerns. In the Matrix federated model, different homeservers host users who can communicate with each other even if they are not connected to the same server. While federation promotes decentralization, it also introduces significant challenges, especially when untrusted servers are involved. Malicious or compromised third-party servers can manipulate messages or metadata, compromising the security and privacy of users. To mitigate these risks, Matrix implements E2EE, ensuring that only end users can access message content. However, potentially malicious servers in the federation are a constant concern as they can harm data integrity or compromise cryptographic key verification. Additionally, Matrix allows users to connect with other communication networks using bridges to interact with external protocols like IRC or XMPP. Despite enhancing interoperability, these bridges introduce new attack vectors, where communication security may be weakened, especially if the target protocol offers a different level of protection than Matrix. Therefore, great interest is expected regarding **Security and Privacy** issues.

Some papers clearly define the threat models used in their experiments and analyses. For instance, Chowdhury et al. [38] paper experiments on six widely used desktop clients using two popular threat elicitation frameworks, STRIDE and LINDDUN.

In our analysis, we assume the presence of a passive third-party adversary. This entity can observe traffic exchanged between clients and servers, attempting to infer message content or communication patterns [19].

In addition, Jacob et al. [11] analyze Byzantine Fault Tolerance and use a simplified threat model for Conflict-Free Replicated Data Type (CRDT). On the other hand, Lu at al. [14] presents a series of discoveries and potential vulnerabilities, including cybersecurity threats in homeservers and compromised confidentiality of encrypted rooms. Other security issues, such as Forward Secrecy, are discussed in the Wichelmann at al. [31], Alwen at al. [39], and Vazquez at al. [32]. Wallez et al. [41] presents a new framework, Comparse, that specifically addresses the security analysis of data formats in cryptographic protocols. Finally, Stajano et al. [44] present a cryptographic challenge-response authentication mechanism to enable portable identities within the Matrix protocol.

In this category, new technologies and approaches are constantly being developed to address emerging challenges. **Networks and Communications** is the second category with the most classified papers. We believe that Matrix's open source license, with example implementations freely licensed without intellectual property restrictions or patent licensing, provides insight and a solid foundation for developing other technologies. This category covers papers that discuss data communication issues, the development of tools and protocols, and network metrics analyses. In some papers, new protocols and tools are proposed. For instance, Rahman at al. [43] introduces Matrix-Mesh, an IM protocol that integrates cloud-dew, client-server, and P2P elements to ensure messaging even without standard connectivity. CrossChat enables communication between different mobile IM apps without modifying the original apps. Cui at al. [42] uses a gateway on mobile devices and UI techniques to capture and exchange messages in real-time. Eisele at al. [34] present TRANSAX, a platform that allows participants to trade energy futures efficiently and privately, helping Distribution System Operators (DSOs). Plan energy needs while ensuring safety while operating in a

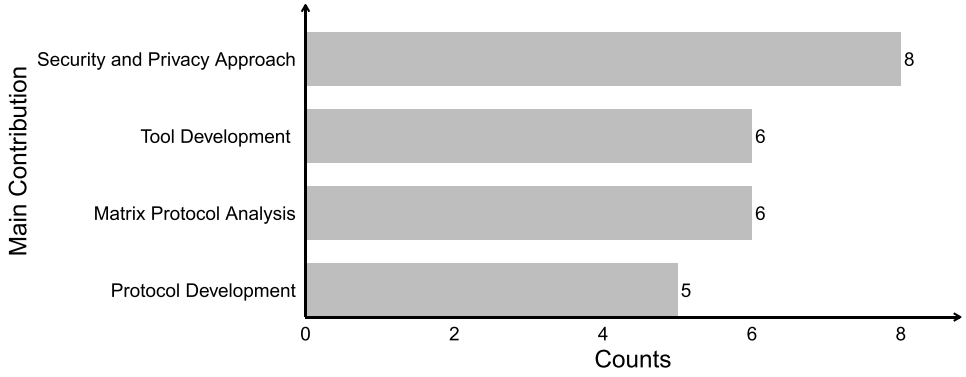


Fig. 9 Primary studies quantified by main contribution

decentralized system provides resilience and must satisfy requirements. Jacob at al. [3] discusses load balancing, asymmetrical communication, and scalability through formal methods analysis or verification of empirical data.

The **Human-Computer Interaction (HCI)** category has two works. Cui at al. [42], extracts the UI tree of IM apps at runtime to avoid modifying the app's software implementation. After extracting the UI tree, CrossChat separates it to identify specific regions for message extraction and sending. Users mark the message window, input box, and send button, which are then used for future operations. This strategy is compared to the use of bridges in the Matrix protocol. In the **Forensic Analysis** category, the author evaluates the Matrix protocol through forensic analysis and provides a detailed report of the communication mechanisms in the form of text, audio, video, and files. The analyses demonstrated that much relevant information can be extracted from the Schipper at al. [2], analyze Matrix protocol and the Riot.im application from a digital forensics perspective. Furthermore, the **Natural Language Processing** category also has one classified article. The author discussed a method for identifying and organizing problem-related information in production chat logs based on employee and company surveys and simulation data to improve digital shop floor management systems. The Natural Language Processing category also explores NLP to enhance interaction between humans and computers. Finally, **Educational Technology** category has one ranked article. The focus is on creating a web-based teaching tool that minimizes bandwidth and power usage by using an event-driven code-casting system instead of video streaming. Brodbeck et al. [35] shows that it supports audio calls and can be extended with modules such as text chat, presentations, Wikipedia, and collaborative text editing. The Matrix chat rooms are used in this tool.

RQ03 - Main contribution

Figure 9 shows the papers according to the main contribution analyzed in the studies.

It is important to note that an article can be classified into multiple categories, reflecting our perception of the work's contribution to a given field of study. Based on these categories, we can observe that most studies focus on the **Security and Privacy Approach** theme, indicating a constant concern with data protection and information integrity. We believe the Matrix Protocol ecosystem requires continuous care to ensure user privacy and promote trust in digital interactions. Operating in decentralized, federated environments while striving for interoperability among network participants presents significant challenges. In decentralized communication networks like the Matrix protocol, security and privacy present unique challenges due to the inherent nature of decentralization. Unlike centralized systems, where a single entity controls data flow, decentralized networks distribute control across multiple independent servers, introducing new vectors for attacks and vulnerabilities. Chowdhury at al. [38], show a critical aspect of securing these networks lies in comprehensive threat modeling that accounts for the diverse attack surfaces presented by federated environments, where varying levels of trust exist between servers. Cryptographic vulnerabilities are a primary concern, as any weaknesses can lead to significant security breaches, such as unauthorized data access or message tampering, compromising the integrity of the entire network. Additionally, in Li et al. [14], privacy considerations are paramount; the decentralized structure makes it challenging to protect user data and communication metadata, which are susceptible to traffic analysis and other privacy-invasive techniques. Federation and trust issues further complicate security efforts, as inconsistencies in security policies across different servers can lead to uneven protection levels, making the network vulnerable to compromised or malicious nodes.

Albrecht et al. [13, 45] argue that addressing security challenges in Matrix requires a robust and evolving cryptographic foundation. Their initial analysis exposed practical weaknesses in key management and authentication workflows, demonstrating that

real-world adversaries could compromise confidentiality and trust in federated deployments. In a subsequent study, the authors expanded this perspective by introducing a formal security model for Matrix, reinforcing the need for continuous evaluation and improvement of cryptographic mechanisms, threat detection strategies, and trust management protocols to ensure secure and privacy-preserving communication in decentralized environments.

Albrecht at al. [13] address these challenges requires a robust and adaptive approach to security, including the continuous evaluation and enhancement of cryptographic methods, threat detection mechanisms, and trust management protocols to safeguard the privacy and security of users in a decentralized environment.

The following papers stand out for their **Tool Development**, each focusing on essential secure and decentralized communication aspects. These four papers demonstrate a significant convergence in developing secure and decentralized communication tools, emphasizing interoperability, efficiency, and privacy. Cui et al. [42] and Brodbeck and Hromada [35] address the need for efficient communication in various contexts, whether through interoperability between messaging apps or by reducing bandwidth consumption in online teaching environments. Both focus on accessibility and adaptability to different platforms and connectivity conditions. Wallez et al. [41], in turn, provides a solid foundation for the security of these communications, ensuring that the cryptographic formats used are verifiable and secure, a central concern also addressed in Eisele et al. [34], which focuses on privacy and transparency in decentralized financial transactions. Together, these contributions highlight the importance of tools that facilitate communication and interaction between diverse systems and ensure users' security and privacy in increasingly decentralized and interconnected environments. Moreover, all the papers demonstrate a concern for balancing technological innovation with simplicity and usability, ensuring that the developed tools are practical and adaptable to the real needs of users. The emphasis on decentralization in these tools reflects a growing trend to move away from centralized systems, vulnerable to failures and attacks, and towards promoting a more resilient and robust architecture. Finally, by integrating aspects of interoperability, security, and efficiency, these tools present new approaches to immediate technical challenges and contribute to advancing secure communication and transaction practices in a rapidly evolving digital world.

In the process of **Protocol Development**, the papers of Rahman et al. [43], Alwen et al. [39], Vazquez et al. [32], and Weidner et al. [9] converge on highlighting security and resilience as crucial components, particularly in decentralized contexts. For instance, Alwen et al. [39] and Weidner et al. [9] explore the use of cryptographic key agreements, such as Elliptic Curve Diffie-Hellman (ECDH), to ensure that communications between parties are secure and authenticated, even in environments where trust among nodes is limited. Additionally, Rahman et al. [43] and Vazquez et al. [32] emphasize the need to design protocols that are not only secure but also efficient and scalable. Rhman et al. [43], for example, proposes a protocol that can adapt to various network conditions, including scenarios with limited connectivity. At the same time, Vazquez et al. [32] addresses lightweight and scalable authentication methods, such as PAKE (Password Authenticated Key Exchange), to prevent system or user overload. All the papers underscore the importance of adopting recognized cryptographic standards, such as the Advanced Encryption Standard (AES) and SHA-256, to ensure the interoperability and reliability of the developed solutions. These studies thus point to a development approach that balances innovation with decentralized systems' stringent security, efficiency, and scalability requirements.

While addressing different aspects of the Matrix protocol, the category **Matrix Protocol Analysis** converges on the importance of analyzing scalability, federation, and interoperability as fundamental pillars for the network's success and security. As seen in Jacob et al. [12] and Jacob et al. [11], the scalability analysis is essential to ensure that Matrix can grow without sacrificing performance or security. Federation, discussed in detail in Franco et al. [46] and Li et al. [14], offers both benefits and challenges, providing resilience through decentralization but also introducing complexity in synchronization and maintaining network cohesion. Interoperability is another critical aspect, enabling different servers and implementations to function seamlessly, as highlighted in Li et al. [14] and Jacob et al. [3]. Finally, the cryptographic vulnerabilities explored in Albrecht et al. [13] demonstrate that despite the advantages offered by federation and interoperability, it is vital that all servers adhere to strict security protocols to avoid compromises. Together, these papers provide a holistic view that connects the technical architecture of Matrix with the practical challenges of maintaining a secure, scalable, and interoperable network in a decentralized environment.

RQ04 - Explored Matrix feature

Figure 10 shows the papers according to the characteristics of the Matrix protocol explored by the papers. Accordingly, we classify the papers' approach into the following characteristics: Group Communication, Security Issues, and Matrix Architecture. The **Group Communication** category encompasses papers that discuss the message exchange mechanisms of the Matrix protocol, addressing issues related to the creation of homeservers,

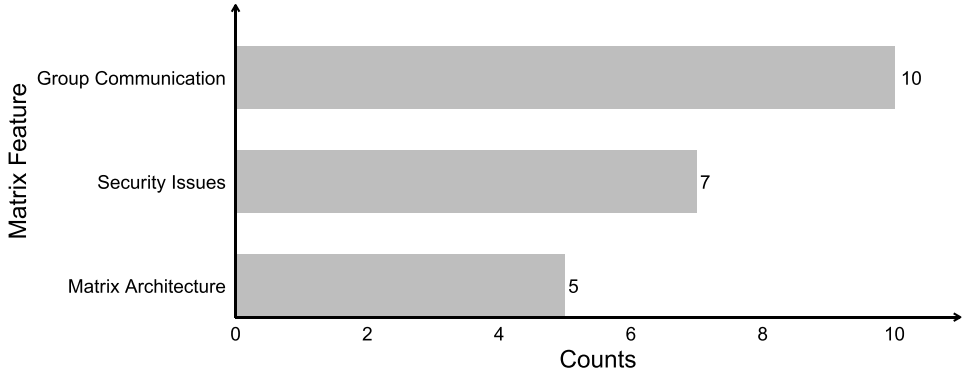


Fig. 10 Primary studies quantified by explored Matrix feature

users, and groups. The Security Issues category classifies papers that address security and privacy issues in the Matrix protocol, such as E2EE and user authentication. On the other hand, the Matrix Architecture category includes studies that analyze the structure and functioning of the Matrix protocol as a whole, exploring aspects related to both Group Communications and Security Issues, among other concerns.

The Group Communication category was the most frequently used to classify works in this research question. We believe this is because a large portion of the paper addresses issues related to federation, such as interoperability and scalability. Additionally, many papers showed great interest in using user chat rooms as a communication model in developing their applications. Jacob et al. [12] directly explore the scalability and centralization issues in server-to-server communication, identifying how the federated structure of Matrix can impact the efficiency of message exchange in groups. In parallel, Brunila et al. [19] and Gibb et al. [40] contribute to the discussion by examining secure group communication, emphasizing E2EE and the application of Matrix in geospatial contexts. Brodbeck and Hromada [35] complement this analysis by adapting the protocol for low-bandwidth environments, ensuring the continuity of group communication under adverse conditions. Cui et al. [42] address interoperability, demonstrating how Matrix can connect different instant messaging platforms, expanding the possibilities for communication between servers of distinct networks. Though focused on extracting problem-related entities in production environments, Müller et al. [33] provides relevant insights into message management and the effectiveness of group communication within Matrix. These studies explore the multiple facets of Group Communication in a Matrix of decentralized communication. When analyzed collectively, these studies offer an exploration of the various dimensions of Group Communication within the Matrix protocol, addressing everything from technical challenges in scalability and server-to-server communication efficiency to critical security and platform interoperability issues. By investigating how Matrix handles group communication in different contexts—whether adapting to low-bandwidth environments, implementing E2EE, or integrating multiple messaging systems—these studies not only elucidate the protocol's strengths and limitations but also pave the way for the development of more robust and scalable solutions. These contributions are important for enhancing decentralized communication, ensuring that Matrix can be effectively applied across a broad spectrum of use cases.

The analyzed papers provide a comprehensive view of **Security Issues** in the Matrix protocol, addressing everything from device compromise protection to the integrity of communications in distributed environments. Wichelmann et al. [31] explore vulnerabilities associated with compromised devices, highlighting the importance of post-compromise security to protect communications within Matrix. Weidner et al. [9] examine decentralized key agreements for secure messaging, which is essential for maintaining security in unstable networks. Chowdhury et al. [38] provide a detailed analysis of threat models in distributed networks, underscoring the need for robust encryption to mitigate attacks. Alwen et al. [39] introduce a fork-resilient protocol, crucial for ensuring the security and integrity of group communications in Matrix. Wallez et al. [41] address the security of cryptographic formats, preventing attacks that could compromise message integrity. Schipper et al. [2] complement this discussion by exploring the forensic analysis of Matrix and the Riot.im client, emphasizing the security challenges related to the acquisition and analysis of encrypted data. Finally, Brunila et al. [19] focus on secure instant messaging with end-to-end encryption, emphasizing the importance of privacy in Matrix communications. Together, these papers lay a foundation for enhancing security practices in Matrix, ensuring more secure and resilient communication in decentralized systems.

The papers investigating **Matrix Architecture** provides a detailed analysis of the structural and functional elements of the Matrix protocol, addressing key concepts such as Event-Based Communication, Client-Server Model, and Federation. Rahman et al. [43] explore the implementation of a hybrid protocol that combines the Client-Server Model with P2P, emphasizing the flexibility required to operate under limited connectivity conditions, where Federation plays a critical role. Li et al. [14] demystify decentralized networks based on Matrix, demonstrating how Federation and Event-Based Communication ensure the scalability and security of server-to-server communication. Albrecht et al. [13] highlight cryptographic vulnerabilities within the Client-Server Model, underscoring the importance of a robust architecture to prevent security breaches. Jacob et al. [11] and Jacob et al. [3] investigate Event-Based Communication and Federation, revealing how these features ensure consistency and resilience in distributed and federated environments. Together, these studies explain how Matrix architecture integrates these elements to create a secure, scalable, and efficient platform capable of operating in various complex scenarios. In addition to explaining these issues, the papers also highlight weaknesses within the Matrix ecosystem, such as the need for improved formal verification and the potential exploitation of cryptographic vulnerabilities, pointing to critical areas for future improvement.

Research challenges and directions

The adoption of the Matrix protocol in decentralized communication systems presents both opportunities and challenges, particularly in ensuring seamless integration and optimized performance across various domains. As the protocol continues to gain traction, several critical challenges emerge, demanding focused research and innovative solutions. These difficulties are a result of the decentralized architectures' inherent complexity, the necessity of strong security and privacy safeguards, the high performance demands of real-time communication, and the need to scale efficiently while preserving interoperability across various platforms and applications. Understanding these challenges is essential for researchers and developers aiming to enhance the Matrix protocol's implementation in real-world environments. In this section, we explore the key research areas that need attention, highlighting existing barriers and potential directions for future studies. By addressing the pressing concerns of security and privacy, optimizing real-time communication processes, tackling scalability and performance issues, and ensuring interoperability, this study aims to provide a foundation for further advancements in decentralized communication technologies. The following subsections present each of these critical areas, offering insights into current research gaps and the directions in which the Matrix protocol's development can be steered to foster broader adoption and integration.

Security and privacy

Cryptographic vulnerabilities: Cryptographic vulnerabilities present a significant challenge in the Matrix protocol, especially when dealing with weak authentication methods, susceptibility to MITM attacks, and protocol confusion issues. Weak authentication can lead to unauthorized access, allowing malicious actors to impersonate legitimate users or systems, thus compromising communication security. Albrecht et al. [13] analyzed attacks like MITM. The susceptibility to MITM attacks is particularly concerning in decentralized networks like Matrix, where the flow of information between servers and clients can be intercepted if encryption is not properly implemented. In these cases, attackers can manipulate or eavesdrop on the communication without the knowledge of the communicating parties. Wichelmann et al. [31] discuss protocol confusion, where different implementations of a protocol handle messages inconsistently, also introduces risks, as attackers may exploit these discrepancies to create vulnerabilities or bypass security measures. These vulnerabilities highlight the need for stronger cryptographic mechanisms and standardized practices across the Matrix ecosystem to ensure robust security. Without addressing these issues, the protocol remains vulnerable to attacks that can undermine its decentralized and secure communication objectives.

Key compromise by malicious servers: According to Chouwdhury et al. [38] in the decentralized architecture of the Matrix protocol, key compromise by malicious servers poses a particularly insidious threat. The federated nature of Matrix means that different servers can communicate and share information, but this also means that a rogue or compromised server could manipulate cryptographic keys to decrypt sensitive messages or even inject malicious content into conversations. This becomes a serious issue when E2EE is in place, as the server could potentially exploit flaws in key distribution or management to gain unauthorized access to private communication. Additionally, a compromised server could disrupt the integrity of key exchange protocols, injecting malicious keys or manipulating sessions to allow an attacker to eavesdrop on or alter communications. As noted by Jacob et al. [3], the lack of a centralized trust authority makes it difficult to prevent such compromises across federated networks, making it essential to develop more robust server verification techniques and key management strategies to mitigate these risks. Without these safeguards, malicious servers can exploit the very openness of Matrix's federated design.

Lack of formal verification in access control: The absence of formal verification in access control

mechanisms is another pressing issue in the Matrix protocol, as it leaves significant room for vulnerabilities and misconfigurations that could be exploited by malicious actors. As observed by Jacob et al. [3], formal verification provides a mathematically rigorous method of proving that access control systems function as intended, ensuring that only authorized users have access to certain resources or communications. Without this layer of assurance, Matrix implementations may contain logic errors or inconsistencies in how access is granted or denied, potentially allowing unauthorized users to gain access to sensitive communications or system resources. This problem is exacerbated by the decentralized nature of Matrix, where different servers and clients may implement access control in slightly different ways, leading to inconsistencies in enforcement. As mentioned in Jacob et al. [11], the lack of formal verification makes it difficult to guarantee that all nodes within a federated system adhere to the same security principles, thereby introducing gaps in security that could be exploited by attackers. Addressing this issue would require developing standardized, formally verified access control mechanisms that can be consistently applied across the entire Matrix ecosystem.

Lack of a unified security model: One of the most significant challenges in the Matrix protocol is the lack of a unified security model across federated servers. In a federated system, different servers are responsible for maintaining their own security policies, which can vary widely in terms of robustness and implementation. This disparity can be exploited by attackers who may find and target weaker servers in the network, thereby compromising the security of the entire federated system. Without a unified security model, it is difficult to ensure consistent protection across all servers, and this inconsistency can lead to security gaps where malicious actors can infiltrate the network. A unified security model would provide a set of standardized security guidelines and protocols that all servers must follow, ensuring a baseline level of security throughout the federated network. The current lack of such a model in Matrix means that security can vary significantly between servers, creating a situation where the overall security of the protocol is only as strong as its weakest link. Standardizing security policies across the federated network is crucial for maintaining the integrity and trustworthiness of the protocol.

Key management issues: Key management is a critical issue in the Matrix protocol, given the use of multiple cryptographic keys for different aspects of communication, such as E2EE, server communication, and identity verification. Managing these keys across a decentralized and federated system introduces significant complexity, as each server and client must maintain and securely store a wide range of keys. The complexity of managing multiple keys increases the risk of key compromise, mismanagement, or loss, any of which could have catastrophic consequences for the security of communications. Furthermore, decentralized key management lacks the centralized oversight that can help ensure keys are handled consistently and securely across the entire system. This makes it difficult to enforce best practices for key rotation, storage, and disposal, which are essential for maintaining long-term security. The current approach to key management in Matrix leaves room for improvement, particularly in terms of developing automated systems that can handle key generation, storage, and rotation in a secure and efficient manner, reducing the burden on users and system administrators while improving overall security.

Potential weaknesses in End-to-end encryption: E2EE is a core feature of the Matrix protocol. However, it does have some weaknesses. The implementation or configuration of encryption algorithms can introduce vulnerabilities that could be exploited by attackers. For instance, if encryption algorithms are not implemented according to best practices or if configuration errors occur during setup, it could leave communications vulnerable to interception or decryption. Additionally, the decentralized nature of Matrix adds further complexity to ensuring that E2EE is consistently applied across all communication channels and devices. Weaknesses in the implementation of E2EE could arise from differences in how various clients or servers handle encryption, leading to situations where certain communications are not as secure as intended. This can be particularly problematic in federated networks, where not all participants may adhere to the same security standards. Addressing potential weaknesses in E2EE requires ongoing audits, standardization of encryption practices, and rigorous testing to ensure that all elements of the Matrix ecosystem are capable of providing strong, consistent encryption for all communications.

Real-time communication

Message latency and synchronization: Message latency is a critical issue in real-time communication, and the decentralized communication model of the Matrix protocol exacerbates these challenges. Unlike centralized systems, where messages pass through a single server, the Matrix protocol involves multiple servers in a federated network, each potentially located in different regions or under varying network conditions. This can lead to significant delays in message delivery, as each server must process and forward messages independently. Latency can be further compounded by the need for synchronization between different servers, which may not always be perfectly aligned due to differences in processing speeds or network conditions. This lack of consistent synchronization can cause messages to be delayed or delivered

out of order, impacting the user experience, especially in environments where timely communication is critical, such as collaborative workspaces or chat systems. Minimizing message latency requires improvements in both server processing capabilities and network efficiency, as well as the development of better synchronization mechanisms to ensure that messages are delivered promptly and in the correct order across all servers.

Maintaining message order: The decentralized nature of the Matrix protocol poses unique challenges in maintaining message order, particularly when dealing with variable network latencies. In real-time communication, ensuring that messages are displayed in the correct order is crucial for preserving the logical flow of conversations. However, in a federated system like Matrix, messages may take different paths through the network, with each path introducing varying amounts of latency. As a result, it is possible for messages sent in rapid succession to arrive out of order, which can cause confusion and disrupt the flow of communication. To address this, the Matrix protocol must rely on mechanisms for reordering messages based on timestamps or other metadata, ensuring that the final display of messages reflects their intended order. This requires careful coordination between servers to track message sequences and resolve any discrepancies caused by network delays. Maintaining message order in decentralized environments remains an ongoing challenge that necessitates continuous refinement of synchronization and latency mitigation techniques.

Quality of Service (QoS) assurance: Ensuring consistent QoS in real-time communication is essential to minimizing delays, jitter, and packet loss, all of which can significantly degrade the user experience. QoS is particularly challenging in decentralized systems like Matrix, where the performance of communication is dependent on a wide variety of factors, including the network conditions of each participating server, the geographic distribution of those servers, and the overall load on the system. Without adequate QoS measures in place, users may experience interruptions in their communication, with messages being delayed or lost entirely. To address this, the Matrix protocol must incorporate mechanisms for monitoring network conditions in real time and adjusting traffic priorities to minimize the impact of congestion or packet loss. Additionally, jitter – the variation in packet arrival times – must be controlled to ensure smooth communication, particularly in scenarios involving multimedia, such as video conferencing or voice calls. Developing more robust QoS frameworks for the Matrix protocol is essential to ensuring reliable real-time communication, regardless of the underlying network conditions.

Integration of Voice over IP (VoIP): The integration of Voice over IP (VoIP) into the Matrix protocol presents a unique set of challenges, particularly with respect to synchronization and latency issues. In real-time voice communication, even small delays or interruptions can result in a poor user experience, as participants may speak over one another or experience gaps in conversation. Latency becomes especially problematic when servers are spread across different regions, as the time it takes for voice packets to traverse the network can introduce noticeable delays. In addition, VoIP requires precise synchronization between servers to ensure that audio streams are transmitted in real time without noticeable interruptions or loss of quality. If synchronization is not properly maintained, voice calls may suffer from echoes, gaps, or other quality issues that make communication difficult. Addressing these challenges requires both optimizing the underlying network infrastructure for real-time audio transmission and developing more sophisticated algorithms for handling synchronization and latency in decentralized environments like Matrix.

Synchronization conflict resolution: Synchronization conflicts arise in the Matrix protocol when multiple servers attempt to process events simultaneously, leading to inconsistencies in the order or content of messages. In real-time communication, these conflicts can be particularly disruptive, as they may result in messages being displayed out of order or entirely missed by one or more participants. Such conflicts are inherent to decentralized systems, where different servers may not always be perfectly synchronized due to varying network conditions or processing speeds. To resolve these conflicts, the Matrix protocol must implement mechanisms for detecting and reconciling discrepancies between servers, ensuring that all participants receive a consistent view of the conversation. This may involve the use of timestamps, sequence numbers, or other metadata to track the order of events and resolve conflicts when they arise. Effective conflict resolution is critical to maintaining the integrity of real-time communication in Matrix, as even small inconsistencies can lead to significant disruptions in the user experience.

Network connection and firewall management: Real-time communication in the Matrix protocol is further complicated by the need to manage network connections across various environments, including traversing firewalls and Network Address Translation (NAT) systems. Firewalls and NATs are commonly used to enhance security by restricting access to certain ports or IP addresses, but they can also introduce challenges for decentralized communication protocols like Matrix, which require seamless peer-to-peer connections. Establishing and maintaining these connections can be difficult when firewalls block necessary traffic or when NAT devices reassign internal IP addresses, disrupting communication. To overcome these obstacles, the Matrix protocol must implement techniques such as hole punching, relays, or

TURN servers, which can help establish connections through firewalls and NATs without compromising security. However, these solutions can introduce additional latency and complexity, making it crucial to balance security and performance in real-time communication scenarios. Effectively managing network connections and firewall traversal is essential to ensuring that Matrix can provide reliable and efficient real-time communication, even in complex network environments.

Scalability and performance

Scalability limitations in federated servers: As the number of federated homeservers in the Matrix protocol grows, scalability limitations become more pronounced, particularly in terms of data synchronization. Federation allows different servers to communicate and share information, but as the number of servers increases, so does the complexity of maintaining consistent data across them. This complexity stems from the need to continuously synchronize messages, events, and user states between servers, which can become increasingly challenging as the network expands. With each additional server, the amount of data that must be processed, stored, and synchronized increases, placing more strain on server resources. Moreover, the decentralized nature of Matrix means that there is no central authority to enforce synchronization standards, making it difficult to ensure that all servers remain up-to-date with the latest data. As a result, scalability becomes a key challenge for the Matrix protocol, requiring improvements in both the underlying architecture and the algorithms used for data synchronization to accommodate the growing number of federated servers without sacrificing performance or consistency.

Impact of data volume on performance: The volume of data generated by the Matrix protocol can have a significant impact on server performance, particularly as the number of events and messages in the network grows. Each message or event must be processed, stored, and synchronized across multiple servers, and as the amount of data increases, so does the load on each server. This can lead to performance degradation, as servers struggle to keep up with the demands of processing large volumes of data in real-time. In particular, chat rooms with high levels of activity or a large number of participants can generate a substantial volume of events, leading to server overload and delays in message delivery. Additionally, the need to store and retrieve historical messages and events adds further strain on server resources, particularly in cases where large amounts of data must be accessed quickly. To mitigate the impact of data volume on performance, the Matrix protocol must implement more efficient data processing and storage mechanisms, as well as optimize the algorithms used for event synchronization and retrieval.

Event synchronization between servers: Event synchronization between federated servers is a fundamental aspect of the Matrix protocol, but it also introduces challenges in terms of performance and consistency. As events are generated in a chat room or communication channel, they must be synchronized across all participating servers to ensure that users receive consistent updates in real-time. However, this continuous synchronization process can cause delays, particularly when there are network latencies or variations in server processing speeds. In some cases, synchronization issues can lead to inconsistencies in message delivery, with certain servers receiving events out of order or experiencing delays in receiving updates. This can disrupt the flow of communication and degrade the user experience, especially in real-time communication scenarios where timely message delivery is critical. Improving event synchronization between servers is essential to enhancing the performance and reliability of the Matrix protocol, particularly as the network grows and the volume of events increases.

Event processing overload: Event processing overload is a common issue in the Matrix protocol, particularly in chat rooms with a large number of participants. As each participant sends messages or generates events, the server must process and synchronize these events with other federated servers, which can lead to delays if the volume of events exceeds the server's processing capacity. This overload can result in delays in message delivery, as servers struggle to keep up with the demands of real-time communication. Additionally, chat rooms with high levels of activity can generate a significant amount of metadata, such as read receipts and typing indicators, which further adds to the processing load. To address event processing overload, the Matrix protocol must implement more efficient event processing algorithms, as well as optimize server resources to handle large volumes of events without causing delays or disruptions in message delivery. By improving event processing efficiency, the Matrix protocol can better accommodate chat rooms with large numbers of participants and high levels of activity.

Delays in message propagation in federated networks: Message propagation delays are a significant challenge in federated networks, such as those used in the Matrix protocol. As messages are sent between servers, they must traverse the network, which can introduce delays due to both network latency and the time required for servers to process and forward messages. In a federated system, each server operates independently, and variations in network conditions or server processing speeds can lead to uneven message delivery times. This can result in certain participants receiving messages later than others,

disrupting the flow of conversation and potentially causing confusion. Additionally, the decentralized nature of Matrix means that there is no central authority to ensure consistent message propagation, making it difficult to guarantee timely message delivery across all servers. To minimize message propagation delays, the Matrix protocol must optimize its routing algorithms and improve network efficiency, ensuring that messages are delivered as quickly as possible, regardless of the underlying network conditions.

Performance of cryptographic processing at scale: The use of E2EE in the Matrix protocol adds a layer of security to communications, but it also introduces significant computational overhead, particularly at scale. E2EE requires that each message be encrypted and decrypted by the sender and recipient, respectively, and as the volume of messages increases, so does the demand for cryptographic processing. This can result in performance issues and high message throughput, as the computational cost of encryption and decryption may become a bottleneck. Furthermore, the decentralized nature of Matrix means that each server must handle its own cryptographic processing, which can lead to inconsistencies in performance across the network. To address the performance challenges associated with cryptographic processing at scale, the Matrix protocol must implement more efficient encryption algorithms and optimize the cryptographic processes used in message transmission. By reducing the computational overhead of E2EE, the Matrix protocol can maintain its security features without sacrificing performance in large-scale environments.

Interoperability

Backward compatibility and protocol complexity: Maintaining backward compatibility in the Matrix protocol is a significant challenge, given the increasing complexity of the protocol as new versions and features are introduced. In a decentralized and federated system like Matrix, where multiple servers and clients interact, ensuring that different versions of the protocol can coexist without disrupting communication is crucial. However, this becomes difficult as older implementations may not support newer features or security improvements, leading to compatibility issues that can affect the overall user experience. The need to maintain compatibility between different versions also increases the complexity of the protocol, as developers must ensure that new changes do not inadvertently break existing functionality. This can lead to situations where certain features are either disabled or only partially supported in older versions, creating a fragmented user experience. Addressing backward compatibility requires careful planning and design, as well as the development of standardized practices to ensure that updates to the protocol can be rolled out smoothly without compromising the stability and usability of the system.

Event synchronization and scalability: Event synchronization is a critical aspect of interoperability in the Matrix protocol, particularly as the network scales. As messages and events are generated, they must be synchronized across all participating servers to ensure that users receive consistent updates in real-time. However, as the network grows and the number of servers increases, synchronization becomes more complex, leading to potential delays in message delivery or partial synchronization. This can result in situations where certain users receive messages later than others, or where messages are lost or duplicated due to incomplete synchronization. Ensuring that event synchronization remains efficient and reliable as the network scales is essential for maintaining the integrity of the communication system. This requires the development of more robust synchronization algorithms and improvements in the underlying infrastructure to handle the increased load without sacrificing performance or reliability.

Federation with non-Matrix systems: Interoperability with non-Matrix systems presents a unique set of challenges, particularly when it comes to federation between different communication platforms. Matrix is designed to be an open and federated protocol, but when interacting with systems that use different data standards or encryption methods, inconsistencies can arise. These differences can lead to data loss, message duplication, or other issues that disrupt the flow of communication. For example, non-Matrix systems may not support the same level of encryption, resulting in messages that are either unencrypted or improperly secured when exchanged between platforms. Additionally, variations in data formats and messaging standards can lead to difficulties in parsing or displaying messages, further complicating cross-platform communication. To address these challenges, Matrix must implement more flexible federation mechanisms that can adapt to the specific requirements of non-Matrix systems while ensuring that security and data integrity are maintained.

Byzantine failures and synchronization guarantees: Byzantine failures present a significant challenge in decentralized systems like Matrix, particularly when it comes to ensuring reliable synchronization across servers. Byzantine failures occur when a server behaves arbitrarily or maliciously, potentially sending incorrect or inconsistent messages to other servers in the network. In the Matrix protocol, which relies on "best-effort" transmission mechanisms, there are limited guarantees that messages will be delivered consistently or in the correct order, especially in the face of Byzantine failures. This can lead to situations where certain users experience delays, receive corrupted messages, or miss messages entirely

due to synchronization issues. To mitigate the impact of Byzantine failures, Matrix must implement stronger synchronization guarantees and fault-tolerant mechanisms that can detect and recover from these failures, ensuring that communication remains reliable even in the presence of malicious or faulty servers. Without these safeguards, the decentralized nature of Matrix leaves the protocol vulnerable to inconsistencies and disruptions in communication.

Identity management and user discovery: User discovery and identity management are critical components of interoperability in the Matrix protocol, but they pose significant challenges, particularly when dealing with users across different domains. Matrix relies on identity servers to help users discover each other, but the decentralized nature of the protocol makes it difficult to maintain a consistent and reliable system for managing identities. Different domains may use different identity servers, and users may have multiple identities across different platforms, leading to confusion and difficulties in discovering and connecting with other users. Additionally, the lack of a unified identity management system can create security vulnerabilities, as attackers may exploit inconsistencies in the way identities are verified or managed across servers. To address these challenges, Matrix must develop more standardized identity management practices and improve user discovery mechanisms to ensure that users can reliably find and connect with each other, regardless of the domain or identity server they are using.

Implementation differences in clients and servers: Interoperability in Matrix is also affected by differences in the implementation of clients and servers, with certain functions or APIs not uniformly supported across the ecosystem. As Matrix is an open protocol, different developers may create their own clients and servers, each with slightly different implementations of the protocol. This can lead to inconsistencies in the way certain features are supported or implemented, resulting in fragmented functionality. For example, a particular API might be fully supported on one server but only partially implemented on another, leading to differences in the user experience depending on which client or server is being used. These implementation differences can also affect compatibility between servers, as certain features may not work as expected when interacting with servers that do not fully support the protocol. To improve interoperability, Matrix must work towards standardizing the implementation of its protocol across different clients and servers, ensuring that key features and APIs are consistently supported throughout the ecosystem.

Conclusion

The Matrix protocol has emerged as a vital technology in decentralized communication, offering significant advantages in terms of security, scalability, and interoperability. This systematic mapping study has provided a comprehensive overview of the current research landscape surrounding the protocol, categorizing studies and analyzing key contributions related to its adoption. Through the examination of 156 papers, narrowed down to 21 primary studies, this research has highlighted trends, research gaps, and opportunities for further exploration. One of the primary findings of this study is that while the Matrix protocol shows great promise in fostering secure, real-time communication across federated servers, several critical challenges remain. Issues related to scalability, event synchronization, and cryptographic overhead need to be addressed to improve performance, particularly in large-scale and high-traffic environments. Moreover, interoperability with non-Matrix systems remains a complex task, requiring more advanced mechanisms to bridge the differences in data standards and encryption protocols.

Security is another major area of focus, as demonstrated by the frequent discussion of E2EE and key management challenges in the reviewed literature. The protocol's reliance on decentralized servers creates opportunities for malicious actors to exploit vulnerabilities, particularly in key compromise scenarios. Strengthening the cryptographic backbone of Matrix and ensuring robust key management across federated servers will be essential for future developments. Interoperability and the need for a unified security model across federated systems are crucial factors that will influence the future adoption of Matrix. The protocol's ability to support various communication platforms highlights its flexibility, but this flexibility must be balanced with consistent security practices and performance standards. The lack of a unified security model across federated servers is an ongoing concern, one that requires urgent attention to prevent inconsistencies and vulnerabilities across the network.

This study also underscores the need for more structured evaluation frameworks to measure the protocol's effectiveness in real-world applications. By identifying research gaps, such as the limited focus on formal verification methods and scalability solutions, this paper provides a roadmap for future research to improve the Matrix protocol's robustness and reliability.

Future research on the Matrix protocol should focus on several key areas. First, formal verification methods for access control and security models need to be developed to enhance resilience against attacks. Scalability improvements are crucial, especially in optimizing event synchronization and message propagation for large-scale

deployments. Additionally, advanced solutions for cross-platform interoperability, particularly in bridging Matrix with non-Matrix systems, must be explored to maintain security and data integrity. Reducing cryptographic overhead while maintaining E2EE is another priority to improve performance. Lastly, enhancing user experience and identity management, including simplifying user discovery and verification, will support broader adoption of the protocol.

Authors' contributions
The authors contributed equally to this work.

Funding
No funding was received for conducting this study.

Data availability
No datasets were generated or analysed during the current study.

Declarations

Competing interests
The authors declare no competing interests.

Received: 4 December 2024 / Accepted: 7 December 2025
Published online: 18 February 2026

References

1. Matrix F (2024) An open network for secure, decentralised communication. https://matrix.org/. Accessed: 2024-08-25
2. Schipper GC, Seelt R, Le-Khac NA (2021) Forensic analysis of Matrix protocol and Riot.Im application. Forensic Sci Int: Digit Investigation 36:301118. https://doi.org/10.1016/j.fsidi.2021.301118
3. Jacob F, Becker L, Grashöfer J, Hartenstein H (2020) Matrix decomposition: analysis of an access control approach on transaction-based DAGs without finality. In Proceedings of the 25th ACM Symposium on Access Control Models and Technologies, SACMAT'20, Association for Computing Machinery. event-place, New York, NY, USA,: Barcelona, Spain., 81–92
4. Alatawi M, Saxena N (2023) SoK: an analysis of end-to-end encryption and authentication ceremonies in secure messaging systems. In Proceedings of the 16th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WiSec'23, New York, NY, USA,: Guildford, United Kingdom., 187–201. Association for Computing Machinery. event-place
5. Foundation M (2022) Mozilla's use of matrix for internal communications. https://blog.mozilla.org/. Accessed: 2024-08-25
6. E (2024) Element: secure messaging app based on matrix protocol. https://element.io/. Accessed: 2024-06-08
7. NHS Digital (2023) How the nhs uses matrix for secure communications. https://digital.nhs.uk/matrix/. Accessed: 2024-10-07
8. Germany G (2022) Berlin's digital transformation: integrating secure communication with matrix protocol. https://govtech.de/berlim-matrix/. Accessed: 2024-10-07
9. Weidner M, Kleppmann M, Hugenroth D, Beresford AR (2021) Key Agreement for decentralized secure group messaging with strong security guarantees. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS'21, New York, NY, USA, 2024–2045. Association for Computing Machinery. event-place: Virtual Event, Republic of Korea
10. Government F (2020) Tchap: the official messaging platform for French civil servants. https://www.tchap.gouv.fr/. Accessed: 2024-08-25
11. Jacob F, Beer C, Henze N, Hartenstein H (2021) Analysis of the Matrix event Graph Replicated data type. IEEE Access 9:28317–28333. https://doi.org/10.1109/ACCESS.2021.3058576
12. Jacob F, Grashöfer J, Hartenstein H (2019 December) A glimpse of the Matrix: scalability issues of a new message-oriented data synchronization middleware. In Proceedings of the 20th International Middleware Conference Demos and Posters, Middleware'19, Association for Computing Machinery, New York, NY, USA, 5–6
13. Albrecht MR, Celi S, Dowling B, Jones D (2023 May) Practically-exploitable cryptographic vulnerabilities in Matrix. In 2023 IEEE Symposium on Security and Privacy (SP), pp 164–181.ISSN:2375-1207
14. Li H, Wu Y, Huang R, Mi X, Hu C, Guo S (2023 December) Demystifying decentralized Matrix communication network: ecosystem and security. In 2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS), pp 260–267. ISSN:2690-5965
15. Project F Fluffychat: a matrix client with a focus on user-friendly design. https://fluffychat.im/. Accessed: 2024-06-08
16. Project H Hydrogen: a lightweight matrix client for web and mobile. https://hydrogen.element.io/. Accessed: 2024-06-08
17. Bernstein DJ (2006) Curve25519: new diffie-hellman speed records. https://cr.yp.to/ecdh/curve25519-20060209.pdf. Accessed: 2024-08-01
18. Moerel L, Timmers P (2021) Digital sovereignty and the open strategic autonomy of europe. SSRN Electron J 10(2139/ssrn.3776179)
19. Brunila M, McConnell M, Grigg S, Appuhn M, Sumner B, Bohman M (2022) Drift: E2EE Spatial feature sharing & instant messaging. In Proceedings of the 6th ACM SIGSPATIAL International Workshop on Location-Based Recommendations, Geosocial Networks and Geoadvertising, LocalRec'22, New York, NY, USA.: Seattle, Washington. Association for Computing Machinery. event-place
20. Hoffman P, McGrew D, Kampanakis P (2022) Transitioning to post-quantum cryptography. Internet Eng Task Force (IETF) Draft IETF draft-ietf-pqc-transition-05
21. Of Standards NI, Technology (2022) Post-quantum cryptography: NIST's plan for the future. Technical report, NIST. Accessed April 2025
22. Project S Schildichat: a matrix client with enhanced user experience features. https://schildi.chat/. Accessed: 2024-06-08
23. Foundation M Synapse: matrix reference homeserver. https://github.com/matrix-org/synapse. Accessed: 2024-06-08
24. Foundation M Dendrite: next-generation matrix homeserver written in go. https://github.com/matrix-org/dendrite. Accessed: 2024-06-08
25. Project C Conduit: lightweight matrix homeserver written in rust. https://conduit.rs/. Accessed: 2024-06-08
26. Johansen C, Mujaj A, Arshad H, Noll J (2021 July) The snowden phone: a comparative Survey of secure instant messaging mobile applications. Secur And Commun Networks 2021:1–30. https://doi.org/10.1155/2021/9965573
27. Blessing J (2023) One protocol to rule them all? On securing interoperable messaging (transcript of discussion). In: Security protocols XXVIII: 28th international workshop. Springer-Verlag. event-place, *Cambridge, UK*, Berlin, Heidelberg: Cambridge, United Kingdom, pp 193–198. *March 27–28*, 2023,Revised Selected Papers
28. Blessing J, Chowdhury PD, Sameen M, Anderson R, Gardiner J, Rashid A (2023) Towards Human-centric endpoint security. In: Stajano F, Matyáš V, Christianson B, Anderson J (eds) Security protocols XXVIII. Springer Nature Switzerland, Cham, pp 211–219
29. Kitchenham B, Charters S et al. (2007) Guidelines for performing systematic literature reviews in software engineering
30. Petersen K, Vakkalanka S, Kuzniarz L (2015) Guidelines for conducting systematic mapping studies in software engineering: an update. Inf And Softw Technol 64:1–18
31. Wichelmann J, Berndt S, Pott C, Eisenbarth T (2021) Help, my Signal has bad Device! Breaking the Signal Messenger's Post-Compromise Security Through a Malicious device. In Detection of Intrusions and Malware, and Vulnerability Assessment: 18th International Conference, DIMVA 2021, Virtual Event, July 14–16, 2021, Proceedings, Springer-Verlag, Berlin, Heidelberg, 88–105
32. Vazquez Sandoval I, Atashpendar A, Lenzini G, Ryan PYA (2021) PakeMail: authentication and key management in decentralized secure email and messaging via PAKE. In: Obaidat MS, Ben-Othman J (eds) E-Business and telecommunications. Springer International Publishing, Cham, pp 102–128
33. Müller M, Lee JU, Frick N, Stangier L, Gurevych I, Metternich J (2021) Extracting problem related entities from production chats to enhance the data base for assistance functions on the shop floor. Procedia CIRP 103:231–236. https://doi.org/10.1016/j.procir.2021.10.037
34. Eisele S, Eghtesad T, Campanelli K, Agrawal P, Laszka A, Dubey A (2021 December) Safe and private forward-trading platform for transactive microgrids. ACM Trans Cyber-Phys. Syst 5(1). https://doi.org/10.1145/3403711
35. Brodbeck F, Hromada DD (2022) February. teacher.Js: a low-bandwidth digital Tool for outdoor online teaching. In 2022 IEEE 5th Eurasian Conference on Educational Innovation (ECEI), pp 47–50

36. Ihle C, Deifuß F, Schubotz M, Gipp B (2022 July) Towards portable identities in the Matrix protocol. In 2022 IEEE 42nd International Conference on Distributed Computing Systems Workshops (ICDCSW), pp 88–89. ISSN:2332-5666
37. Griggio CF, Nouwens M, Klokmose CN (2022) Caught in the network: the impact of WhatsApp's 2021 privacy policy update on users' messaging app ecosystems. In Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, CHI'22, Association for Computing Machinery. event-place, New York, NY, USA.: New Orleans, USA
38. Chowdhury PD, Sameen M, Blessing J, Boucher N, Gardiner J, Burrows T, Anderson R, Rashid A (2023 May) Threat models over space and time: a case study of E2EE messaging applications. arXiv:2301.05653 [cs
39. Alwen J, Mularczyk M, Tselekounis Y (2023) Fork-resilient continuous group key Agreement. In Advances in Cryptology – CRYPTO 2023: 43rd Annual International Cryptology Conference, CRYPTO 2023, Springer-Verlag. event-place, Santa Barbara, CA, USA,, Berlin, Heidelberg, August 20–24, 2023, Proceedings, Part IV, 396–429
40. Gibb R, Madhavapeddy A, Crowcroft J (2023) Where on Earth is the Spatia name system? In Proceedings of the 22nd ACM Workshop on Hot Topics in Networks, HotNets'23, New York, NY, USA,: Cambridge, MA, USA., 79–86. Association for Computing Machinery. event-place
41. Wallez T, Protzenko J, Bhargavan K (2023) Comparse: provably secure formats for cryptographic protocols. In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS'23, New York, NY, USA,: Copenhagen, Denmark., 564–578. Association for Computing Machinery. event-place
42. Cui M, Gu C, Nan G (2023 March) CrossChat: instant messaging across different apps on mobile devices |. In Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing
43. Rahman M, Wang Y, De D (2024) Implementation of dew-inspired matrix-mesh communication protocol. In: Roy S (ed) Dew computing: the sustainable IoT perspectives. Springer Nature Singapore, Singapore, pp 105–115. https://doi.org/10.1007/978-981-99-4590-0_5
44. Blessing J, Chowdhury PD, Sameen M, Anderson R, Gardiner J, Rashid A, Stajano F (2023) Towards Human-centric endpoint security. In: Matyáš V, Christianson B, Anderson J (eds) Security protocols XXVIII, vol 14186. Springer Nature Switzerland. Series Title: Lecture Notes in Computer Science. Cham, pp 211–219. https://doi.org/10.1007/978-3-031-43033-6_21
45. Albrecht MR, Dowling B, Jones D (2024 May) Device-oriented group messaging: a formal cryptographic analysis of matrix core. In 2024 IEEE Symposium on Security and Privacy (SP), IEEE
46. Franco M, Gaggi O, Guidi B, Michienzi A, Palazzi CE (2023 August) A decentralised messaging system robust against the unauthorised forwarding of private content. Future Gener Comput Syst 145:211–222. https://doi.org/10.1016/j.future.2023.03.025

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.